

Mnemis: Dual-Route Retrieval on Hierarchical Graphs for Long-Term LLM Memory

Zihao Tang, Xin Yu*, Ziyu Xiao, Zengxuan Wen, Zelin Li, Jiaxi Zhou, Hualei Wang, Haohua Wang, Haizhen Huang, Denvy Deng, Feng Sun, Qi Zhang

Microsoft

*Corresponding Author: xinyu2@microsoft.com

AI Memory, specifically how models organizes and retrieves historical messages, becomes increasingly valuable to Large Language Models (LLMs), yet existing methods (RAG and Graph-RAG) primarily retrieve memory through similarity-based mechanisms. While efficient, such System-1-style retrieval struggles with scenarios that require global reasoning or comprehensive coverage of all relevant information. In this work, We propose Mnemis, a novel memory framework that integrates System-1 similarity search with a complementary System-2 mechanism, termed Global Selection. Mnemis organizes memory into a base graph for similarity retrieval and a hierarchical graph that enables top-down, deliberate traversal over semantic hierarchies. By combining the complementary strength from both retrieval routes, Mnemis retrieves memory items that are both semantically and structurally relevant. Mnemis achieves state-of-the-art performance across all compared methods on long-term memory benchmarks, scoring 93.9 on LoCoMo and 91.6 on LongMemEval-S using GPT-4.1-mini.

🌐 **Project:** <https://github.com/microsoft/Mnemis>
📅 **Date:** 6 Jan 2026

1. Introduction

With the rapid advancement of Large Language Models (LLMs), there is a growing trend to integrate memory mechanisms to support long-term interactions (Lewis et al., 2020; Ouyang et al., 2025; Behrouz et al., 2024; Wu et al., 2025a,b; Tang et al., 2024; Liang et al., 2025). As LLMs shift from text generators to persistent interactive agents, the ability to organize and retrieve past interactions becomes increasingly valuable. The prevailing research paradigm is based on retrieval-augmented generation (RAG). Inspired by human episodic memory (Tulving et al., 1972), these methods (e.g. SeCom (Pan et al., 2025), Memory-R1 (Yan et al., 2025)) explicitly store historical messages (i.e., *Episodes*) and retrieve only the most relevant pieces (Arslan et al., 2024; Lewis et al., 2020). This design alleviates the computational and latency issues of long-context models and keeps the input compact and focused. However, its effectiveness critically depends on retrieval quality.

Recent work on graph-based RAG (Graph-RAG) extends RAG by incorporating concepts from semantic memory (Tulving et al., 1972). Graph-RAG extracts memory segments, e.g. *Entities* (key figures, objects, or concepts) and *Edges* (events or relationships connecting Entities) and organizes memory into a structured graph, as exemplified by methods such as GraphRAG (Edge et al., 2024), Nemori (Nan et al., 2025), Mem0 (Chhikara et al., 2025), and Zep (Rasmussen et al., 2025). These methods highlight essential information and enable more effective and semantically meaningful retrieval.

Although Graph-RAG-based methods mark an important step toward structured memory, their retrieval remains largely similarity-driven, selecting Episodes, Entities, or Edges via text matching (BM25) or embedding similarity (cosine). This approach is fast and effective, and resembles the System-1 process in dual-process theory (Kahneman, 2011), but becomes limited when queries require global reasoning or comprehensive coverage of all relevant information. Although recent research has explored iterative generation of sub-queries to mitigate this issue, such methods still fall short for questions that require a broader perspective (Wang

Mnemis: 面向长期LLM记忆的层次图双路径检索

Zihao Tang, Xin Yu*, ZiyuXiao, Zengxuan Wen, Zelin Li, Jiaxi Zhou, Hualei Wang, Haohua Wang, 黄海珍, Denvy Deng, Feng Sun, Qi Zhang

微软

*Corresponding Author: xinyu2@microsoft.com

AI记忆，特别是模型如何组织和检索历史信息，对大语言模型（LLMs）而言正变得越来越有价值。然而，现有方法（RAG和图检索增强生成）主要通过基于相似性的机制来检索记忆。虽然高效，但这种系统一风格的检索在需要全局推理或全面覆盖所有相关信息的场景中表现不佳。在这项工作中，我们提出了Mnemis，一种新颖的记忆框架，它将系统一相似性搜索与一种互补的系统二机制——全局选择——相结合。Mnemis将记忆组织成一个用于相似性检索的基础图和一个层次图，后者支持在语义层次上进行自上而下、深思熟虑的遍历。通过结合两种检索路径的互补优势，Mnemis能够检索出在语义和结构上都相关的记忆项。Mnemis在所有对比方法中，于长时记忆基准测试上取得了最先进的性能，使用GPT-4.1-mini在LoCoMo上得分为93.9，在LongMemEval-S上得分为91.6。

🌐 **项目:** <https://github.com/microsoft/Mnemis>
📅 **日期:** 2026年1月6日

1. 引言

随着大语言模型的快速发展，集成记忆机制以支持长期交互的趋势日益明显（Lewis等人，2020；Ouyang等人，2025；Behrouz等人，2024；Wu等人，2025a,b；Tang等人，2024；Liang等人，2025）。随着大语言模型从文本生成器转变为持久的交互式智能体，组织和检索过往交互的能力变得越来越有价值。主流的研究范式基于检索增强生成。受人类情景记忆的启发（Tulving等人，1972），这些方法（例如SeCom（Pan等人，2025）、Memory-R1（Yan等人，2025））显式地存储历史消息（即事件片段），并仅检索最相关的片段（Arslan等人，2024；Lewis等人，2020）。这种设计缓解了长上下文模型的计算和延迟问题，并保持了输入的紧凑性和聚焦性。然而，其有效性关键取决于检索质量。

近期基于图的检索增强生成（图检索增强生成）研究通过融入语义记忆的概念（Tulving 等人, 1972）扩展了检索增强生成。图检索增强生成提取记忆片段，例如实体（关键人物、对象或概念）和边（连接实体的事件或关系），并将记忆组织成结构化图，相关方法示例如 GraphRAG（Edge 等人, 2024）、Nemori（Nan 等人,2025）、Mem0（Chhikara 等人, 2025）和 Zep（Rasmussen 等人, 2025）。这些方法能突出关键信息，实现更有效且语义丰富的检索。

尽管基于图检索增强生成的方法标志着向结构化记忆迈出了重要一步，但其检索过程在很大程度上仍是相似性驱动的，通过文本匹配（BM25）或嵌入相似度（余弦相似度）来选择事件片段、实体或边。这种方法快速有效，类似于双过程理论中的系统1过程（卡尼曼, 2011年），但当查询需要全局推理或全面覆盖所有相关信息时，其局限性就显现出来。尽管最近的研究探索了通过迭代生成子查询来缓解此问题，但此类方法对于需要更广阔视角的问题仍然力有未逮（Wang

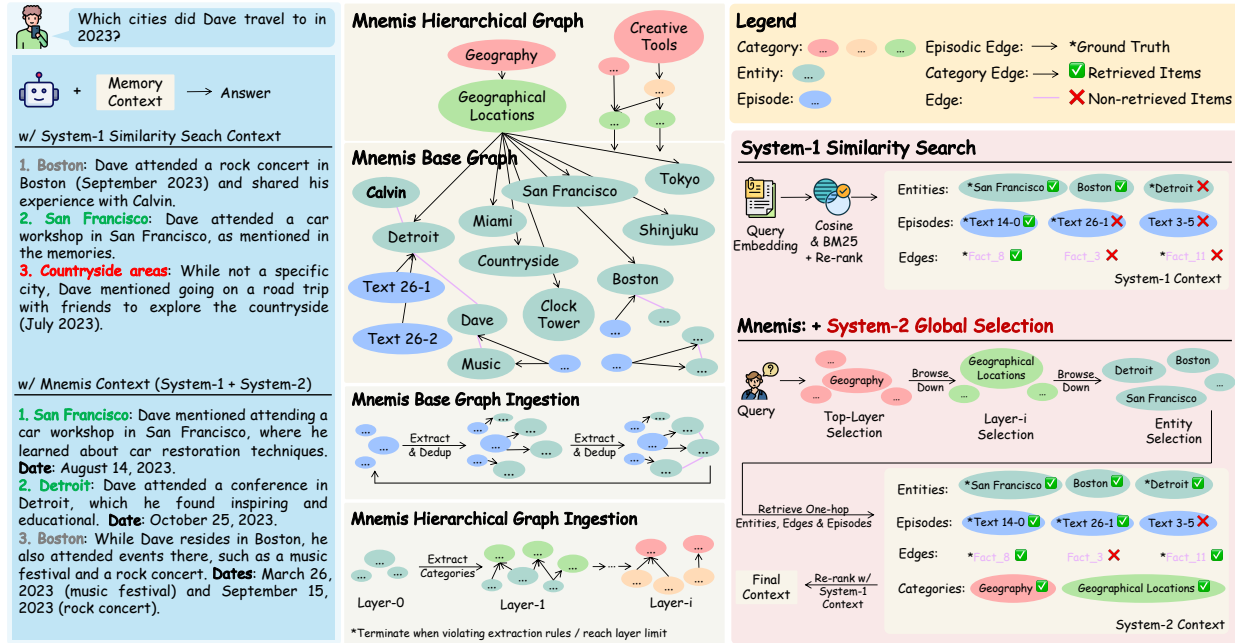


Figure 1: Framework of Mnemis together with the workflow of base graph ingestion, hierarchical graph ingestion and search. Left is a real case from LoCoMo.

et al., 2025; Jin et al., 2025). For example, consider the query "Which cities did Dave travel to in 2023?" from LoCoMo Benchmark (Maharana et al., 2024), as shown in fig. 1. The mention "attended a conference in Detroit." is buried in a long message and has only a weak semantic relation to the user query. Moreover, generating effective sub-queries is challenging as the model lacks a global view of the memory to determine how the original query should be meaningfully expanded.

Recalling how human approach such questions, it can be naturally addressed using a semantic hierarchy. We can begin with a high-level concept (e.g. city), enumerate all the cities we have visited and verify them one-by-one. This kind of solution operates over a global view of memory and naturally avoids the need for sub-query generation, reflecting a structured process characteristic of System-2 reasoning (Kahneman, 2011).

Inspired by this observation, we propose an analogous mechanism, called *Global Selection*, which constructs a hierarchical graph that provides a complete, global, and structured view of the entire memory, mimicking human semantic hierarchies. It allows models to perform top-down, deliberate memory scanning within it. In this example, Global Selection can start from the top layer and follow the path "Geography" → "Geographical Locations" → "Detroit" to retrieve the relevant information.

In practice, real-world queries often benefit from combining both the System-1 and System-2 processes, as they operate through different retrieval patterns. Motivated by this, we present Mnemis, a novel and effective framework to organize and retrieve AI memory. Mnemis comprises two storage components: a base graph and a hierarchical graph, and two corresponding retrieval routes: System-1 similarity search and System-2 global selection. The base graph, similar to prior Graph-RAG designs, extracts Entities and Edges from history texts (Episodes) to support similarity-based retrieval. We refine the extraction pipeline to increase extraction fields and improve extraction quality. In contrast, the hierarchical graph prompts LLMs to categorize Entities into higher-level Categories through bottom-up. This process follows three key principles: (1) Minimum Concept Abstraction: each Category should faithfully capture the shared features of its child nodes. It should be specific enough to be informative, yet sufficiently general to support abstraction; (2) Many-to-Many Mapping: one child node can be assigned to multiple Categories to represent its different semantic facets; and (3) Compression Efficiency Constraint: one Category must contain at least n children and higher layers must contain no more Categories than lower layers (applied from layer 2 onward).

When a query arrives, the similarity search route conducts a semantic search based on embeddings and text

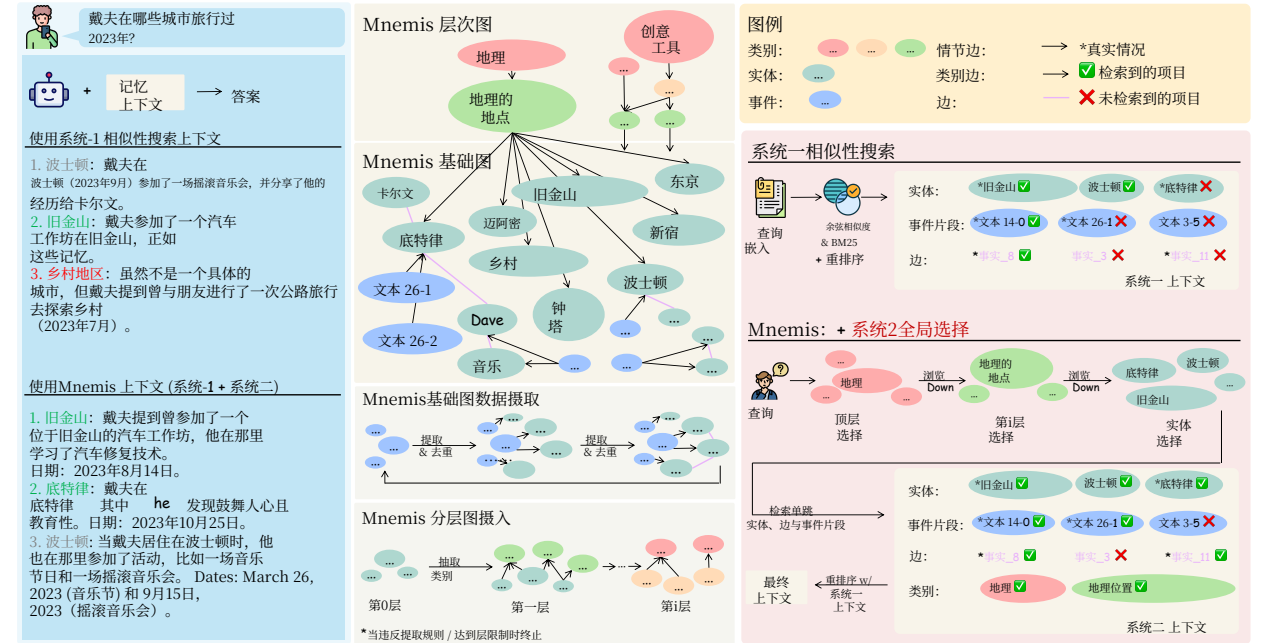


图1: Mnemis框架，以及基础图摄取、分层图摄取和搜索的工作流程。左侧是来自LoCoMo的一个真实案例。

et al., 2025; Jin et al., 2025)。例如，考虑来自 LoCoMo 基准测试（“戴夫在2023年去过哪些城市？”来自 LoCoMo 基准测试 (Maharana et al., 2024) 的查询，如图1所示。提及“在底特律参加了一个会议。”的信息埋藏在一段长消息中，并且与用户查询仅有微弱的语义关联。此外，生成有效的子查询具有挑战性，因为模型缺乏对记忆的全局视图，无法确定原始查询应如何进行有意义的扩展。

回想人类如何处理这类问题，利用语义层级结构可以自然地解决。我们可以从一个高层概念（例如城市）开始，枚举所有我们访问过的城市并逐一验证。这种解决方案基于对记忆的全局视图进行操作，自然地避免了生成子查询的需要，这反映了系统2推理（卡尼曼，2011年）的结构化过程特征。

受此观察启发，我们提出了一种类似的机制，称为全局选择，它构建了一个层次图，提供了对整个记忆的完整、全局且结构化的视图，模仿了人类语义层次。它允许模型在其中执行自上而下、深思熟虑的记忆扫描。在此示例中，全局选择可以从顶层开始，沿着路径“地理” → “地理位置” → “底特律”来检索相关信息。

在实践中，现实世界的查询通常受益于结合系统一和系统二这两种过程，因为它们通过不同的检索模式运作。受此启发，我们提出了Mnemis，一个新颖且有效的框架，用于组织和检索AI记忆。Mnemis包含两个存储组件：一个基础图和一个层次图，以及两条对应的检索路径：系统一相似性搜索和系统二全局选择。基础图类似于先前的图检索增强生成设计，从历史文本（事件片段）中抽取实体和边，以支持相似性检索。我们改进了提取管道，以增加提取字段并提高提取质量。相比之下，层次图提示大语言模型通过自下而上的方式将实体分类到高层级类别中。此过程遵循三个关键原则：(1) 最小概念抽象：每个类别应忠实地捕捉其子节点的共同特征。它应足够具体以提供信息，同时又足够通用以支持抽象；(2) 多对多映射：一个子节点可以被分配到多个类别，以表示其不同的语义层面；以及(3) 压缩效率约束：一个类别必须至少包含 n 个子节点，且更高层级包含的类别数量不得超过更低层级（从第2层开始应用）。

当查询到达时，相似性搜索路由会基于嵌入向量和文本进行语义搜索，

similarity, while the global search performs a top-down selection through the hierarchical graph, layer by layer. Down to the lowest level, the LLM first selects all relevant entities and then retrieves all edges, entities and episodes connected to them. These two routes capture complementary signals: System-1 provides fine-grained semantic similarity evidence, while System-2 retrieves structurally relevant items that may be semantically distant yet relationally important. By combining and re-ranking the union of both routes, Mnemis achieves SOTA performance across all compared methods on long-term memory benchmarks, scoring 93.9 on LoCoMo and 91.6 on LongMemEval-S using GPT-4.1-mini. Our contributions can be summarized as below:

- We introduce Mnemis, a novel framework that integrates System-1 similarity search with System-2 global selection to perform both semantic retrieval and deliberate, top-down reasoning over memory;
- We improve the base graph extraction and construct a hierarchical graph for global selection, guided by Minimum Concept Abstraction, Many-to-Many Mapping, and Compression Efficiency Constraint to maintain hierarchical quality;
- We perform comprehensive experiments to demonstrate the effectiveness of Mnemis. Mnemis achieves SOTA performance across all compared methods on long-term memory benchmarks, scoring 93.9 on LoCoMo and 91.6 on LongMemEval-S using GPT-4.1-mini.

2. Mnemis Methodology

To achieve effective memory organization, Mnemis constructs two major components: a base graph and a hierarchical graph and two key memory retrieval mechanisms: System-1 Similarity Search and System-2 Global Selection. We implement Mnemis based on Graphiti¹.

2.1. Base Graph

The base graph stores historical messages and captures detailed information, enabling the model to perform System-1 Similarity Search, *i.e.*, retrieving semantically relevant histories. It consists of four components: Episodes, Entities, Edges and Episodic Edges.

Episodes. Each episode is a piece of raw historical text. It is encoded into an `episode_embedding` for similarity-based retrieval. Its timestamp is recorded at `valid_at`.

Entities. An entity is any concrete person, organization, place, object, event, or well-defined concept. Each entity includes `name`, `summary`, `tag`, and `episode_idx`. The `summary` provides a concise contextual description, the `tag` specifies its type or role, and `episode_idx` tracks the episodes it appears. We encode `name` and `summary` into corresponding embeddings for flexible search.

Edges. An edge is a verifiable statement describing a meaningful relationship, action, or state involving one or more specified entities within a defined temporal or contextual scope. Each edge connects two entities through a `fact`, which is encoded as a `fact_embedding`. Additionally, `valid_at` and `invalid_at` specify the time span during which the edge is considered valid.

Episodic Edges. An episodic edge links entities to all episodes where they appear. It is utilized during global search to retrieve all episodes associated with selected entities.

The ingestion of the base graph is conducted incrementally: new inputs are first formatted into Episodes. Based on their timestamps, recent Episodes will be retrieved to provide additional context. During extraction, the LLM first identifies entity names from both the current and recent Episodes, followed by a reflection process to capture omitted entity names. These names are then de-duplicated against existing entities in memory, using a combination of full-text search and similarity search over the `name_embedding`. After de-duplication, each entity’s `summary`, `tag`, and `episode_idx` are extracted according to the episode context. Subsequently, Edges are extracted using both Episodes and Entities as contextual inputs, followed by reflection and de-duplication steps analogous to those used in entity extraction. To ensure the robustness of base graph ingestion, we forcibly rule-extract the speaker as an entity.

¹<https://github.com/getzep/graphiti>

而全局搜索则通过层次图自上而下逐层进行选择。向下至最底层，大语言模型首先选择所有相关实体，然后检索所有与之相连的边、实体和事件片段。这两条路由捕获了互补的信号：系统一提供细粒度的语义相似性证据，而系统二则检索那些可能在语义上较远但在关系上重要的、结构相关的项目。通过结合并重排序两条路由的并集，Mnemis在所有对比方法的长时记忆基准测试中实现了SOTA性能，使用GPT-4.1-mini在LoCoMo上得分为93.9，在LongMemEval-S上得分为91.6。我们的贡献可总结如下：

- 我们提出了Mnemis，这是一个新颖的框架，它集成了系统一相似性搜索与系统2全局选择，以执语义检索以及对记忆进行深思熟虑的、自上而下的推理；
- 我们改进了基础图抽取，并在最小概念抽象、多对多映射和压缩效率约束的指导下，构建了用于全局选择的层次图，以维持层次质量；
- 我们进行了全面的实验以证明Mnemis的有效性。Mnemis在所有对比方法中，于长时记忆基准测试上均取得了SOTA性能，使用GPT-4.1-mini在LoCoMo上得分为93.9，在LongMemEval-S上得分为91.6。

2. Mnemis 方法

为了实现有效的记忆组织，Mnemis构建了两个主要组件：一个基础图和一个层次图，以及两个关键的记忆检索机制：系统一相似性搜索和系统2全局选择。我们基于Graphiti¹实现了Mnemis。

2.1. 基础图

基础图存储历史消息并捕获详细信息，使模型能够执行系统一相似性搜索，即，检索语义相关的历史记录。它由四个组件构成：事件片段、实体、边和事件边。

事件片段。每个事件片段是一段原始历史文本。它被编码为 `episode_embedding` 以进行相似性检索。其时间戳记录在 `valid_at`。

实体。实体是任何具体的人、组织、地点、对象、活动或明确定义的概念。每个实体包含 `name`、`summary`、`tag`和 `episode_idx`。`summary` 提供简洁的上下文描述，`tag` 指定其类型或角色，`episode_idx` 则追踪其出现的事件片段。我们将`name` 和 `summary` 编码为相应的嵌入向量，以实现灵活的搜索。

边。边是一种可验证的陈述，描述在定义的时序或上下文范围内，涉及一个或多个指定实体的有意义的关系、动作或状态。每条边通过 `fact`连接两个实体，该 `fact`被编码为 `fact_embedding`。此外，`valid_at` 和 `invalid_at` 指定了该边被视为有效的时间跨度。

事件边。一条事件边将实体链接到它们出现的所有事件片段。它在全局搜索期间被用来检索与所选实体相关的所有事件片段。

基础图摄取是增量进行的：新输入首先被格式化为事件片段。根据其时间戳，将检索近期的事件片段以提供额外的上下文。在抽取过程中，大语言模型首先从当前及近期的事件片段中识别实体名称，随后通过一个反思过程来捕获被忽略的实体名称。接着，通过结合在 `name_embedding`上的全文搜索和相似性搜索，将这些名称与记忆中的现有实体进行去重。去重后，根据事件片段上下文，抽取每个实体的 `summary`、`tag`和 `episode_idx`。随后，使用事件片段和实体作为上下文输入来抽取边，并遵循与实体抽取类似的反思和去重步骤。为确保基础图摄取的鲁棒性，我们强制将说话者作为实体进行规则抽取。

¹<https://github.com/getzep/graphiti>

2.2. Hierarchical Graph

The hierarchical graph abstracts Entities (layer 0) into multi-level Categories, enabling the LLM to perform System-2 Global Selection. The structure consists of two components, as shown in fig. 2.

Category Nodes (Categories). A category represents an abstract, high-level concept derived from lower-layer categories (or entities at layer 0). It shares the same core fields as an Entity, with an additional attribute `layer` indicating its position within the hierarchical graph.

Category Edges. A category edge links a higher-layer category to its child nodes (either lower-layer categories or entities). These edges define the hierarchical organization of the graph and support the top-down traversal process in global selection.

The ingestion of hierarchical graph is governed by three key design principles:

Minimum Concept Abstraction. While categories are intended to capture the shared semantics of their child nodes, we explicitly prompt the LLM to perform *minimal abstraction*. The resulting category should remain sufficiently specific to preserve informative detail, leaving room for broader generalizations at higher layers.

Many-to-Many Mapping. Unlike conventional tree-structured hierarchies, Mnemis permits lower-layer nodes to belong to multiple higher-layer categories. This design allows the hierarchy to represent different semantic facets of each node, enabling retrieval from multiple perspectives depending on the query.

Compression Efficiency Constraint. To ensure the efficiency of System-2 Global Selection, the hierarchy is regulated by two complementary mechanisms: (1) the *compression ratio n* and (2) the *node count reduction rule*, which takes effect from layer 2 onward.

The compression ratio constrains the hierarchy at the category level. Each category must contain at least n child nodes. An exception is made for nodes that cannot be naturally merged with others; such nodes are directly promoted to the next layer as standalone categories, encouraging meaningful aggregation while preventing overly fine-grained or trivial categories.

The node count reduction rule, in contrast, constrains the hierarchy at the layer level: each upper layer must contain no more nodes than the layer beneath it, ensuring progressive abstraction across layers. If this rule is violated, e.g., when multiple nodes are promoted directly without merging and the result layer is oversized, the ingestion process is terminated to maintain hierarchical balance.

Guided by the principles above, the hierarchical graph is constructed layer by layer. At layer i , all nodes from layer $i - 1$ are first retrieved. Category names are then generated, and lower-layer nodes are assigned to these categories using their names and tags as contextual information. The construction process terminates when either the compression efficiency constraints are violated or the maximum layer limit is reached.

Practical Scalability. During base graph ingestion, we include a fallback design that explicitly extracts speakers as dedicated entities. Accordingly, We aggregate them into a layer-1 Speaker category in the hierarchical graph as shown in section D. Speaker is further organized into higher-level categories, introducing additional retrieval paths and thereby improving recall and reducing sensitivity to local categorization errors. For example, queries such as user's allergy can be resolved either through health-related categories or through the speaker branch. When the base graph is updated, the hierarchical graph should be updated accordingly. Currently, we periodically rebuild the hierarchical graph for simplicity and leave optimization for future work.

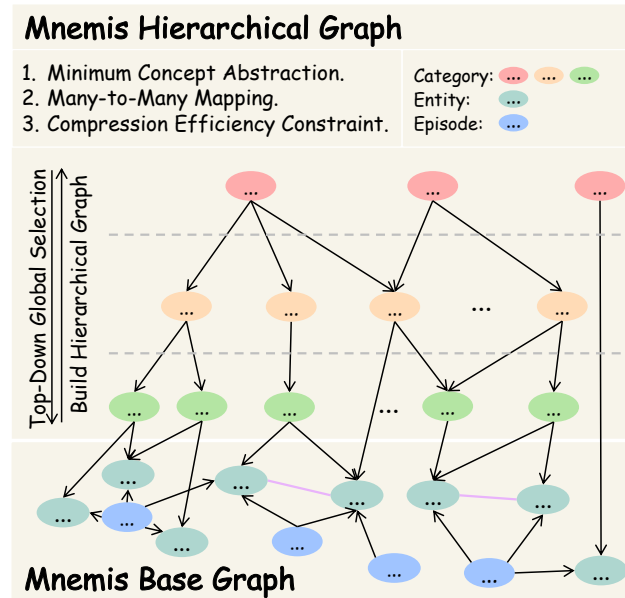


Figure 2: Mnemis Hierarchical Graph Overview.

2.2. 层次图

层次图将实体（第0层）抽象为多级类别，使大语言模型能够执行系统2全局选择。该结构由两个组件组成，如图2所示。

类别节点（类别）。 类别表示从较低层类别（或第0层的实体）衍生出的抽象高层概念。它与实体共享相同的核心字段，并具有一个额外属性 `layer`，用于指示其在层次图中的位置。

类别边。 一条类别边将一个较高级别的类别与其子节点（较低层级的类别或实体）连接起来。这些边定义了图的层次组织结构，并支持全局选择中的自上而下遍历过程。

层次图的数据注入遵循三个关键设计原则：

最小概念抽象。 虽然类别的目的是捕捉其子节点的共享语义，但我们明确提示大语言模型执行最小抽象。生成的类别应保持足够的特异性以保留信息细节，为更高层级的更广泛概括留出空间。

多对多映射。 与传统的树状结构层级不同，Mnemis 允许较低层级的节点属于多个较高级别的类别。这种设计使得层级结构能够表示每个节点的不同语义层面，从而允许根据查询从多个角度进行检索。

压缩效率约束。 为确保系统2全局选择的效率，层级结构受两种互补机制调节：(1) 压缩比 n 和 (2) 从第2层开始生效的节点数量减少规则。

压缩比在类别级别上约束着层级结构。每个类别必须至少包含 n 个子节点。对于无法自然地与其他节点合并的节点，则作为例外处理；此类节点将直接提升至下一层，作为独立类别，从而鼓励有意义的聚合，同时防止产生过于细粒度或无意义的类别。

相比之下，节点数量减少规则在层级级别上约束着层级结构：每个上层包含的节点数不得超过其下层，以确保跨层级的渐进抽象。如果违反此规则，例如，当多个节点未经合并直接提升，导致结果层规模过大时，数据注入过程将终止，以维持层次平衡。

遵循上述原则，层次图被逐层构建。在第 i 层，首先检索来自第 $i - 1$ 层的所有节点。随后生成类别名称，并使用其名称和标签作为上下文信息，将下层节点分配到这些类别中。当违反压缩效率约束或达到最大层数限制时，构建过程即告终止。

实用可扩展性。 在基础图摄取过程中，我们包含了一个后备设计，明确将说话者提取为专用实体。相应地，我们将它们聚合到层次图中的第一层说话者类别中，如章节D所示。说话者被进一步组织到高层级类别中，这引入了额外的检索路径，从而提高了召回率并降低了对局部分类错误的敏感性。例如，诸如用户过敏之类的查询，既可以通过健康相关类别解决，也可以通过说话者分支解决。当基础图更新时，层次图也应相应更新。目前，为简化起见，我们定期重建层次图，并将优化工作留待未来。

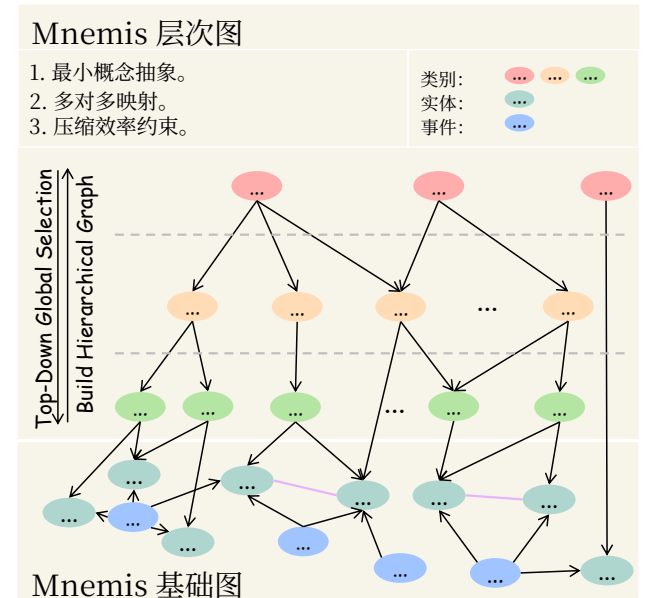


图 2: Mnemis 层次图概览。

2.3. Memory Retrieval Mechanisms

Basically, Mnemis contains two major memory retrieval routes: System-1 Similarity Search and System-2 Global Selection. Given a user query, Mnemis retrieves Episodes, Entities and Edges, formatting them into a context and prompts LLM to get the final answer.

System-1 Similarity Search. This route retrieves the top- k Episodes, Entities, and Edges, providing fast and effective retrieval based on semantic similarity. It operates through two complementary methods: embedding search, which retrieves relevant items by computing cosine similarity between the query embedding and the corresponding embeddings (`summary_embedding` for Entities, `fact_embedding` for Edges, and `episode_embedding` for Episodes); and full-text search, which retrieves relevant components using BM25 over textual content (`content` for Episodes, `name` and `summary` for Entities, and `fact` for Edges). These two results are then merged and re-ranked using reciprocal rank fusion (RRF) (Cormack et al., 2009), which computes a fusion score by summing the reciprocals of each candidate’s ranks and orders candidates in descending score. Episodes, Entities, and Edges are re-ranked separately. A higher $RRFScore(x)$ corresponds to a higher rank for the candidate. The re-ranked results are then truncated to the top- k items according to the predefined search budget.

System-2 Global Selection. This route enables deliberate, top-down exploration of memory through the hierarchical graph. Because the process is primarily structure-driven and the selection at each layer is fully determined by the LLM, no strict top- k constraint is applied. Starting from the top layer, the LLM uses category names and tags to select relevant Categories based on the user query and progressively browses down the hierarchy layer by layer. At the lowest level, all relevant entities are first selected. Mnemis then retrieves all episodes and edges directly connected to these entities, along with the entities linked through those edges.

Re-ranking. After executing both retrieval routes, we apply a re-ranking model to leverage their complementary strengths.² Episodes, Entities (Categories), and Edges are re-ranked separately. These items are then reformatted into a unified memory context and provided to the answer model, together with the user query, to generate the final response.

Practical Scalability. To reduce the extra inference cost introduced by Global Selection, we employ **early stopping**, which terminates traversal once the model determines that all child nodes under the current branch are relevant, avoiding unnecessary deeper exploration.

3. Experiments

3.1. Experiment Setups

Datasets. We evaluate Mnemis on two well-known AI memory benchmarks: LoCoMo (Maharana et al., 2024) and LongMemEval-S (Wu et al., 2024a). LoCoMo consists of long-term conversations from 10 users, with each user contributing approximately 600 turns across 32 sessions, totaling around 16K tokens on average. The dataset contains roughly 2,000 questions spanning five diverse categories: Single-Hop, Multi-Hop, Temporal, Open-Domain, and Adversarial. LongMemEval-S comprises 500 sessions, with each session containing one question and roughly 115K tokens, designed to evaluate five core memory abilities: information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention.

Baselines. We compare Mnemis against the following baselines: LangMem³, MemOS (Li et al., 2025a), Mem0 (Chhikara et al., 2025), Zep (Rasmussen et al., 2025), Nemori (Nan et al., 2025), PreMem (Kim et al., 2025), EverMemOS⁴, EMem-G (Zhou, 2025) using GPT-4o-mini or GPT-4.1-mini as the backend model for memory building and question answering. We directly use their reported performance. In addition, we include two supplementary baselines: Full Context, which feeds the entire conversation history to the model, and RAG, which retrieves only episodes while keeping all other settings identical to Mnemis. We also identified several other comparable baselines; however, due to missing details such as the backbone model and hyperparameter

2.3. 记忆检索机制

基本上，Mnemis包含两条主要的记忆检索路径：系统一相似性搜索和系统二全局选择。给定一个用户查询，Mnemis会检索事件片段、实体和边，将它们格式化为上下文，并提示大语言模型以获取最终答案。

系统一相似性搜索。此路径检索前 k 个事件片段、实体和边，基于语义相似性提供快速有效的检索。它通过两种互补的方法运作：嵌入搜索，通过计算查询嵌入与相应嵌入向量之间的余弦相似度来检索相关项（`summary_embedding`用于实体，`fact_embedding`用于边，以及`episode_embedding`用于事件片段）；以及全文搜索，通过BM25算法在文本内容上检索相关组件（`content`用于事件片段，`name`和`summary`用于实体，以及`fact`用于边）。然后使用互逆排序融合（RRF）（Cormack等人，2009）合并并重排序这两种结果，该方法通过累加每个候选者排名的倒数来计算融合分数，并按分数降序排列候选者。事件片段、实体和边分别进行重排序。更高的RRF分数（ x ）对应候选者更高的排名。然后根据预定义的搜索预算，将重排序后的结果截断为前 k 项。

系统2全局选择。此路径支持通过层次图进行深思熟虑、自上而下的记忆探索。由于该过程主要由结构驱动，且每一层的选择完全由大语言模型决定，因此不应用严格的前 k 约束。从顶层开始，大语言模型利用类别名称和标签，根据用户查询选择相关类别，并逐层向下浏览层级结构。在最底层，首先选择所有相关实体。随后，Mnemis检索与这些实体直接相连的所有事件片段和边，以及通过这些边链接的实体。

重排序。在执行完两条检索路径后，我们应用一个重排序模型来利用它们的互补优势。²事件片段、实体（类别）和边被分别重排序。这些项目随后被重新格式化为统一的记忆上下文，并与用户查询一起提供给答案模型，以生成最终响应。

实用可扩展性。为了降低全局选择引入的额外推理成本，我们采用**早停**机制，一旦模型确定当前分支下的所有子节点都相关，便终止遍历，避免不必要的更深层探索。

3. 实验

3.1. 实验设置

数据集。我们在两个知名的AI记忆基准测试上评估Mnemis：LoCoMo（Maharana et al., 2024）和LongMemEval-S（Wu et al., 2024a）。LoCoMo包含来自10位用户的长期对话，每位用户贡献了约600轮对话，跨越32个会话，平均总计约16K个词元。该数据集包含约2,000个问题，涵盖五个不同的类别：单跳、多跳、时序、开放域和对抗性。LongMemEval-S包含500个会话，每个会话包含一个问题及约115K个词元，旨在评估五种核心记忆能力：信息抽取、多会话推理、时序推理、知识更新和弃权。

基线方法。我们将Mnemis与以下基线方法进行比较：LangMem³、MemOS（Li等人，2025a）、Mem0（Chhikara等人，2025）、Zep（Rasmussen等人，2025）、Nemori（Nan等人，2025）、PreMem（Kim等人，2025）、EverMemOS⁴、EMem-G（Zhou，2025），它们使用GPT-4o-mini或GPT-4.1-mini作为记忆构建和问答的后端模型。我们直接使用其报告的性能。此外，我们包含两个补充基线：完整上下文，它将整个对话历史输入模型；以及RAG，它仅检索事件片段，同时保持所有其他设置与Mnemis相同。我们还发现了其他几个可比较的基线；然而，由于缺少诸如骨干模型和超参数等细节

²As System-2 produces unordered results, we cannot directly apply an RRF re-ranker as in System-1.

³<https://github.com/langchain-ai/langmem>

⁴<https://github.com/EverMind-AI/EverMemOS/>

²由于系统二产生无序结果，我们无法像在系统一中那样直接应用RRF重排序器。

³<https://github.com/langchain-ai/langmem>⁴<https://github.com/EverMind-AI/EverMemOS/>

Table 1: Detailed performance (LLM-as-a-Judge score) on LoCoMo by question type. Following the common practice, Category 5 (Adversarial) is excluded from the results.

LLM	Methods	Multi-Hop	Temporal	Open-Domain	Single-Hop	Overall
#Questions		282	321	96	841	1540
GPT-4o-mini	Full Context	66.8	56.2	48.6	83.0	72.3
	RAG	59.9	62.9	63.5	73.5	68.2
	LangMem	52.4	24.9	47.6	61.4	51.3
	MemOS	64.3	73.2	55.2	78.4	73.3
	Mem0	60.3	50.4	40.6	68.1	61.3
	Zep	50.5	58.9	39.6	63.2	58.5
	Nemori	65.3	71.0	44.8	82.1	74.4
	EMem-G	74.7	76.0	57.3	82.3	78.0
	Mnemis	89.7	77.6	79.2	95.7	89.8
GPT-4.1-mini	Full Context	77.2	74.2	56.6	86.9	80.6
	RAG	64.9	76.6	67.7	76.5	73.8
	LangMem	71.0	50.8	59.0	84.5	73.4
	Mem0	68.2	56.9	47.9	71.4	66.3
	Zep	53.7	60.2	43.8	66.9	61.6
	Nemori	75.1	77.6	51.0	84.9	79.5
	PREMem	61.0	74.8	46.9	66.2	65.8
	EverMemOS	91.1	89.7	70.8	96.1	92.3
	EMem-G	79.6	80.8	71.7	90.5	85.3
	Mnemis	91.8	90.3	82.3	96.2	93.3
	Mnemis ($k=30$)	92.9	90.7	79.2	97.1	93.9

settings, we report their results only in section B.

Hyperparameters. Following Nemori (Nan et al., 2025), we limit the number of retrieved episodes in the answer prompt to $\text{top-}k = 10$, while entities (including categories) and edges are limited to $\text{top-}2k = 20$. We use Qwen3-Embedding-0.6B as the embedding model, with the embedding dimension fixed at 128 due to storage constraints. The re-ranker model used in the main experiments is Qwen3-Reranker-8B (Zhang et al., 2025). We use neo4j⁵ as the backend database. Across all experiments, the grader model is consistently GPT-4.1-mini to ensure accurate scoring.

Metrics. We employ LLM-as-a-Judge score (0/1) for evaluation and adopt the official judger prompt for each dataset. Following previous methods, Category 5 of LoCoMo is excluded from the final score.

3.2. Experiment Results

The results can be found in tables 1 and 2. Below, we provide detailed discussion on the results.

Full-context models alone are insufficient for long-horizon AI memory. Across all settings, we observe a clear divergence between Full Context and RAG as context length grows. In LoCoMo, where the average context is roughly 16K tokens, which is well within the optimal operating window of modern LLMs (128K), the Full Context model remains competitive with most baselines. However, this behavior changes dramatically in LongMemEval-S, whose average context length reaches 115K tokens. As the input approaches or exceeds the model’s practical context limit, the Full Context model consistently degrades. This contrast suggests an important implication for long-term memory: real deployments must support months or years of accumulated interaction history, far beyond what can be reliably handled by a single forward pass over the full context. Thus, relying solely on the model’s native context window without any additional memory management or retrieval mechanisms is insufficient for long-horizon, persistent AI memory systems.

Mnemis consistently outperforms all baselines. With limited and aligned context budget (10 episodes, 20

表1: 按问题类型划分的LoCoMo详细性能 (LLM作为评判者的分数)。遵循惯例, 类别5 (对抗性) 已从结果中排除。

LLM	方法	多跳	时序	开放域	单跳	总体
#问题		282	321	96	841	1540
GPT-4o-mini	完整上下文	66.8	56.2	48.6	83.0	72.3
	RAG	59.9	62.9	63.5	73.5	68.2
	LangMem	52.4	24.9	47.6	61.4	51.3
	MemOS	64.3	73.2	55.2	78.4	73.3
	Mem0	60.3	50.4	40.6	68.1	61.3
	Zep	50.5	58.9	39.6	63.2	58.5
	Nemori	65.3	71.0	44.8	82.1	74.4
	EMem-G	74.7	76.0	57.3	82.3	78.0
	Mnemis	89.7	77.6	79.2	95.7	89.8
GPT-4.1-mini	完整上下文	77.2	74.2	56.6	86.9	80.6
	RAG	64.9	76.6	67.7	76.5	73.8
	LangMem	71.0	50.8	59.0	84.5	73.4
	Mem0	68.2	56.9	47.9	71.4	66.3
	Zep	53.7	60.2	43.8	66.9	61.6
	Nemori	75.1	77.6	51.0	84.9	79.5
	PREMem	61.0	74.8	46.9	66.2	65.8
	EverMemOS	91.1	89.7	70.8	96.1	92.3
	EMem-G	79.6	80.8	71.7	90.5	85.3
	Mnemis	91.8	90.3	82.3	96.2	93.3
	Mnemis ($k=30$)	92.9	90.7	79.2	97.1	93.9

设置, 我们仅在章节 B 中报告其结果。

超参数. 遵循 Nemori (Nan 等人, 2025) 的做法, 我们将答案提示中检索到的片段数量限制为 $\text{top-}k = 10$, 而实体 (包括类别) 和边则限制为 $\text{top-}2k = 20$ 。我们使用 Qwen3-Embedding-0.6B 作为嵌入模型, 由于存储限制, 嵌入维度固定为 128。主要实验中使用的重排序模型是 Qwen3-Reranker-8B (Zhang 等人, 2025)。我们使用 neo4j⁵ 作为后端数据库。在所有实验中, 评分模型始终为 GPT-4.1-mini, 以确保评分的准确性。

指标. 我们采用 LLM作为评判者的分数 (0/1) 进行评估, 并为每个数据集采用官方评判提示。遵循先前的方法, LoCoMo 的类别5 被排除在最终分数之外。

3.2. 实验结果

结果可见于表 1 和 2。下文我们将对结果进行详细讨论。

仅靠全上下文模型不足以实现长视野AI记忆。 在所有设置中, 我们观察到随着上下文长度的增长, 完整上下文与检索增强生成之间存在明显的性能差异。在LoCoMo中, 平均上下文长度约为16K个词元, 这完全处于现代大语言模型 (128K) 的最佳操作窗口内, 此时完整上下文模型仍能与大多数基线方法竞争。然而, 在平均上下文长度达到115K个词元的LongMemEval-S中, 这一情况发生了显著变化。当输入接近或超过模型的实际上下文限制时, 完整上下文模型的性能持续下降。这种对比为长期记忆揭示了一个重要启示: 实际部署必须支持数月或数年累积的交互历史, 这远远超出了对完整上下文进行单次前向传播所能可靠处理的范围。因此, 仅依赖模型自身的上下文窗口, 而不借助任何额外的内存管理或检索机制, 对于构建长视野、持久性AI记忆系统是不够的。

Mnemis 始终 y 在所有基线方法中表现最优。 在有限且对齐的上下文预算下 (10个事件片段、20个

⁵<https://neo4j.com/>

⁵<https://neo4j.com/>

Table 2: Detailed performance (LLM-as-a-Judge score) on LongMemEval-S, categorized by question type: single-session-user (SSU), multi-session (MS), single-session-preference (SSP), temporal reasoning (TR), knowledge update (KU), and single-session-assistant (SSA).

LLM	Methods	SSU	MS	SSP	TR	KU	SSA	Overall
#Questions		70	133	30	133	78	56	500
GPT-4o-mini	Full Context	78.6	38.3	6.7	42.1	78.2	89.3	55.0
	RAG	88.6	47.4	<u>70.0</u>	63.2	70.5	91.1	67.2
	Mem0	91.4	66.2	34.0	63.9	74.4	<u>96.4</u>	71.1
	Zep	92.9	47.4	53.3	54.1	74.4	75.0	63.2
	Nemori	88.6	51.1	46.7	61.7	61.5	83.9	64.2
	EMem-G	87.0	<u>73.6</u>	32.2	<u>74.8</u>	94.4	87.5	<u>77.9</u>
	Mnemis	97.1	76.7	90.0	83.5	<u>92.3</u>	100.0	87.2
GPT-4.1-mini	Full Context	85.7	51.1	16.7	60.2	76.9	98.2	65.6
	RAG	82.9	54.9	86.7	67.7	80.8	94.6	72.6
	PREMem	92.9	57.1	36.7	59.4	84.6	12.5	60.8
	Mem0	94.3	66.9	86.7	75.9	87.2	96.4	80.8
	Nemori	90.0	55.6	86.7	72.2	79.5	92.9	74.6
	EverMemOS	100.0	78.5	<u>96.7</u>	71.2	87.2	78.6	82.0
	EMem-G	94.8	82.6	50.0	83.7	94.4	87.5	<u>84.9</u>
	Mnemis	<u>98.6</u>	86.5	100.0	86.5	<u>93.6</u>	100.0	91.6

Table 3: Detailed LLM cost of Mnemis on LoCoMo using GPT-4.1-mini, reported as the **overall** number of prompt tokens, completion tokens, and end-to-end runtime for processing the entire dataset. Runtime depends heavily on database latency and parallelism configuration; the reported values are for reference only, and we will continue to optimize it for greater efficiency.

Stage	#Prompt Tokens	#Completion Tokens	E2E Runtime(s)
Base Graph Ingestion	3.87×10^7	1.06×10^6	1111.40
Hierarchical Graph Ingestion	1.39×10^7	9.27×10^5	3873.26
Global Selection	1.37×10^6	1.21×10^5	3637.65

entities, and 20 edges), Mnemis achieves consistently superior performance across both benchmarks. For relatively easier tasks that are solvable within a single session or via single-hop reasoning, such as Single-Hop in LoCoMo and single-session-user, single-session assistant, and single-session-preference in LongMemEval-S, Mnemis reaches near-saturated scores. More importantly, on the challenging categories that require multi-hop evidence aggregation or complex temporal or event reasoning, Mnemis shows substantially larger margins over all baselines. These results demonstrate Mnemis’s strong ability to organize and retrieve memory.

We also report the overall LLM cost of Mnemis when using GPT-4.1-mini to test LoCoMo in table 3.

3.3. Ablation Study

To further assess the effectiveness of Mnemis, we conduct comprehensive experiments from four perspectives: (1) the influence of System-1 and System-2 routes on the final results; (2) the effect of backend models (re-ranker, embedding model, LLM); (3) the impact of the top- k parameter. For simplicity, these experiments are conducted on LoCoMo using GPT-4.1-mini.

3.3.1. Influence of System-1 and System-2 Routing

As stated in previous sections, System-1 Similarity Search provides a fast, heuristic retrieval mechanism based on similarities, while System-2 Global Selection performs a more structured and reflective selection process. To evaluate their individual and combined contributions, we compare three configurations: using only System-1,

表2: LongMemEval-S上的详细性能（LLM作为评判者的分数），按问题类型分类：单会话用户（SSU）、多会话（MS）、单会话偏好（SSP）、时序推理（TR）、知识更新（KU）和单会话助手（SSA）。

LLM	方法	SSU	MS	SSP	TR	KU	SSA	总体
#问题		70	133	30	133	78	56	500
GPT-4o-mini	完整上下文	78.6	38.3	6.7	42.1	78.2	89.3	55.0
	RAG	88.6	47.4	<u>70.0</u>	63.2	70.5	91.1	67.2
	Mem0	91.4	66.2	34.0	63.9	74.4	<u>96.4</u>	71.1
	Zep	92.9	47.4	53.3	54.1	74.4	75.0	63.2
	Nemori	88.6	51.1	46.7	61.7	61.5	83.9	64.2
	EMem-G	87.0	<u>73.6</u>	32.2	<u>74.8</u>	94.4	87.5	<u>77.9</u>
	Mnemis	97.1	76.7	90.0	83.5	<u>92.3</u>	100.0	87.2
GPT-4.1-mini	完整上下文	85.7	51.1	16.7	60.2	76.9	98.2	65.6
	RAG	82.9	54.9	86.7	67.7	80.8	94.6	72.6
	PREMem	92.9	57.1	36.7	59.4	84.6	12.5	60.8
	Mem0	94.3	66.9	86.7	75.9	87.2	96.4	80.8
	Nemori	90.0	55.6	86.7	72.2	79.5	92.9	74.6
	EverMemOS	100.0	78.5	<u>96.7</u>	71.2	87.2	78.6	82.0
	EMem-G	94.8	<u>82.6</u>	50.0	83.7	94.4	87.5	<u>84.9</u>
	Mnemis	<u>98.6</u>	86.5	100.0	86.5	<u>93.6</u>	100.0	91.6

表3: Mnemis在LoCoMo上使用GPT-4.1-mini的详细大语言模型成本，报告为处理整个数据集所需的**总体**提示词元数量、补全词元数量以及端到端运行时间。运行时间在很大程度上取决于数据库延迟和并行配置；报告的值仅供参考，我们将持续优化以提升效率。

阶段	#提示词元	#补全词元	端到端运行时间（秒）
基础图摄取	3.87×10^7	1.06×10^6	1111.40
分层图摄取	1.39×10^7	9.27×10^5	3873.26
全局选择	1.37×10^6	1.21×10^5	3637.65

实体和20条边），Mnemis 在两个基准测试中均取得了持续领先的性能。对于相对较简单、可在单次会话内或通过单跳推理解决的任务，例如 LoCoMo 中的单跳推理，以及 LongMemEval-S 中的单会话用户、单会话助手和单会话偏好，Mnemis 的得分接近饱和。更重要的是，在需要多跳证据聚合或复杂时序或事件推理的挑战性类别上，Mnemis 相对于所有基线方法均展现出显著更大的优势。这些结果证明了 Mnemis 在组织和检索记忆方面的强大能力。

我们还报告了使用 GPT-4.1-mini 测试 LoCoMo 时 Mnemis 的总体大语言模型成本，见表格3。

3.3. 消融研究

为了进一步评估 Mnemis 的有效性，我们从四个角度进行了全面的实验：(1) 系统一和系统-2 路径对最终结果的影响；(2) 后端模型（重排序器、嵌入模型、大语言模型）的效果；(3) top- k 参数的影响。为简化起见，这些实验均在 LoCoMo 上使用 GPT-4.1-mini 进行。

3.3.1. 系统一和系统-2 路径的影响

如前面章节所述，系统一相似性搜索提供了一种基于相似性的快速启发式检索机制，而系统2全局选择则执行一种更具结构性和反思性的选择过程。为了评估它们各自及组合的贡献，我们比较了三种配置：仅使用系统一、

using only System-2, and using both jointly. For System-1, we further analyze four settings: (1) System-1 RAG: use retrieved episodes only; (2) System-1 Graph: use retrieved entities and edges only; (3) System-1 RAG + Graph: use episodes, entities and edges jointly; and (4) System-1 Re-ranked: the same to (3) but replace RRF re-ranker with Qwen3-Reranker-8B.

Table 4: Detailed performance (LLM-as-a-Judge score) on LoCoMo by question type.

Settings	Multi-Hop	Temporal	Open-Domain	Single-Hop	Overall
System-1 RAG	64.9	76.6	67.7	76.5	73.8
System-1 Graph	84.8	62.6	74.0	88.6	81.6
System-1 RAG + Graph	85.1	84.7	75.0	<u>93.7</u>	<u>89.1</u>
System-1 Re-ranked	<u>88.7</u>	<u>85.0</u>	75.0	92.4	<u>89.1</u>
System-2 Only	88.1	78.5	<u>79.5</u>	92.0	87.7
System-1 + System-2	91.8	90.3	82.3	96.2	93.3

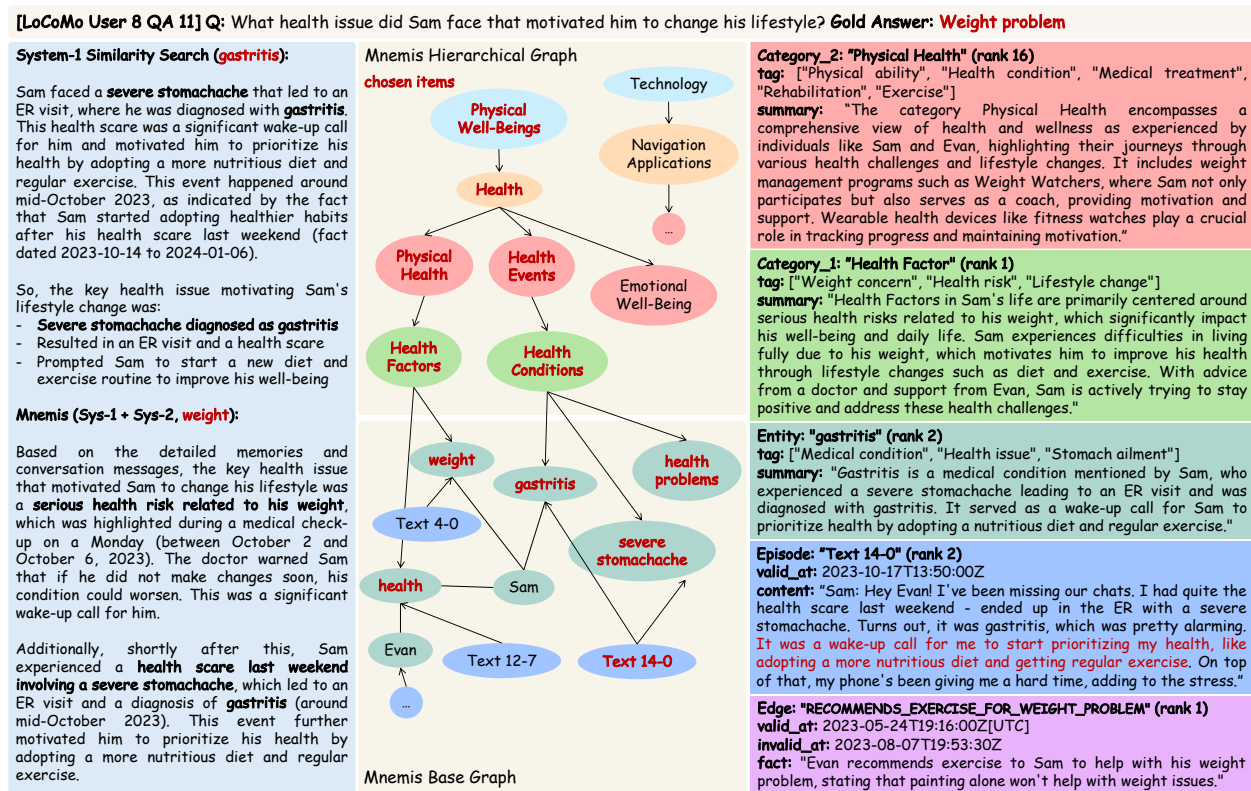


Figure 3: Mnemis win case on LoCoMo benchmark. While similarity search fixates on the surface-level cause gastritis, Mnemis successfully identifies the underlying root cause, namely that Sam is overweight. This condition leads to the gastritis and motivates him to change his lifestyle.

The results can be found in table 4. System-1 Graph slightly wins System-1 RAG as the entities and edges are more condensed and informative compared to raw episodes. However, the compression comes at the cost of certain information loss, where episodes could compensate it. System-1 RAG + Graph hence achieves higher scores. The introduction of a re-ranking model affects performance in some sub-categories, but the overall score remains comparable to System-1 RAG + Graph. This indicates that the performance gain of System-1 + System-2 is not primarily driven by the re-ranking model, but instead stem from global selection.

According to the essence of the System-2 route, not all user queries are suitable for this process. We expect it to perform well on enumerative problems (e.g., "find all items that ...") but weakly on temporal problems.

仅使用系统二，以及两者联合使用。对于系统一，我们进一步分析了四种设置：(1) 系统-1 RAG：仅使用检索到的片段；(2) 系统-1 图：仅使用检索到的实体和边；(3) 系统-1 RAG +图：联合使用片段、实体和边；以及 (4) 系统1重排序：与(3)相同，但将RRF重排序器替换为Qwen3-Reranker-8B。

表4：按问题类型划分的LoCoMo详细性能（LLM作为评判者的分数）。

设置	多跳	时序	开放域	单跳	总体
系统-1 RAG	64.9	76.6	67.7	76.5	73.8
系统-1 图	84.8	62.6	74.0	88.6	81.6
系统-1 RAG + 图	85.1	84.7	75.0	<u>93.7</u>	<u>89.1</u>
系统1重排序	<u>88.7</u>	<u>85.0</u>	75.0	92.4	<u>89.1</u>
仅系统-2	88.1	78.5	<u>79.5</u>	92.0	87.7
系统-1 +系统-2	91.8	90.3	82.3	96.2	93.3

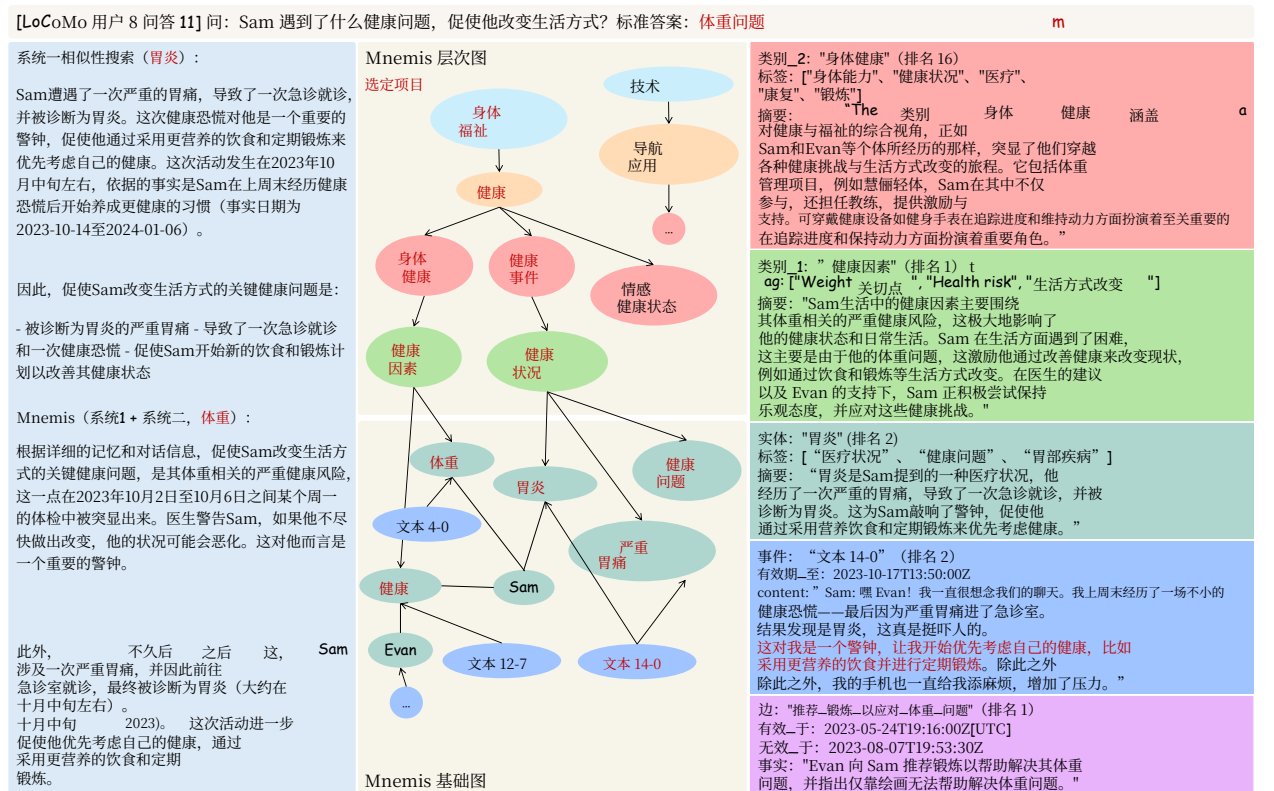


图 3: Mnemis 在 LoCoMo 基准测试中的获胜案例。虽然相似性搜索只关注表面原因胃炎，但 Mnemis 成功识别了根本原因，即 Sam 超重。这一状况导致了胃炎，并促使他改变生活方式。

结果见表格 4。系统-1 图略微优于系统-1 RAG，因为其实体和边相比原始事件片段更为凝练且信息丰富。然而，这种压缩是以某些信息丢失为代价的，而事件片段可以弥补这一点。系统-1 RAG +图因此获得了更高的分数。引入重排序模型会影响某些子类别的性能，但总体得分与系统-1 RAG + 图相当。这表明系统-1 +系统-2 的性能提升并非主要由重排序模型驱动，而是源于全局选择。

根据系统-2 路径的本质，并非所有用户查询都适合此流程。我们预期它在枚举问题（例如，“查找所有满足...的项目”）上表现良好，但在时序问题上表现较弱。

Since the search query remains unchanged during the route, it may identify some key points but fail to capture the full sequence of temporal events. In the experiments, about 90.06% (1387 / 1540) queries obtain the results, we hence report the average score on these valid queries. The results match our expectations. With both routes combined, all categories are improved, leveraging the complementary strengths of them.

To be more intuitive, we present Mnemis win cases from the LoCoMo and LongMemEval-S benchmarks to demonstrate the effectiveness of introducing System-2 Global Selection. As illustrated in fig. 3, when addressing the query "What health issue did Sam face that motivated him to change his lifestyle?", similarity-based search could only retrieve "gastritis", merely a surface reason found in Episode Text 14-0. In contrast, equipped with Global Selection, we browse the hierarchical graph through "Physical Well-Beings" → "Health" → "Health Events" → "Health Conditions" → "gastritis" to locate the surface cause, and through "Physical Well-Beings" → "Health" → "Physical Health" → "Health Factors" → "weight" to locate the essential root cause. Along the search path, intermediate Categories, such as "Physical Health" and "Health Factors" naturally aggregate relevant information from their descendants in summary, which further enriches the retrieved context. For case from LongMemEval-S, please refer to section A.

3.3.2. Impact of top-k

The top-k parameter is introduced to balance answer cost and accuracy. In the main experiments, we use top-k = 10, meaning that 10 episodes, 20 entities, and 20 edges are included in the context to generate the final answer. To evaluate the impact of top-k, we vary it across values of 5, 10, 30, and 50, and conduct experiments under the same configuration as section 3.3.1.

The results are shown in fig. 4 with details in table 10. Reducing top-k from 10 to 5 leads to a clear performance drop in most settings, indicating that a top-k of 5 is insufficient to capture all the evidence needed for user queries. System-1 RAG is especially sensitive to this reduction, particularly on Multi-Hop questions where diverse evidence across multiple parts of the history is required. In contrast, System-1 Graph, which retrieves more information-dense entities and edge, is less affected. System-1 RAG + Graph, which combines episodes, entities, and edges, shows more stable performance, and applying the re-ranker in System-2 Only or System-1 + System-2 further minimizes fluctuations.

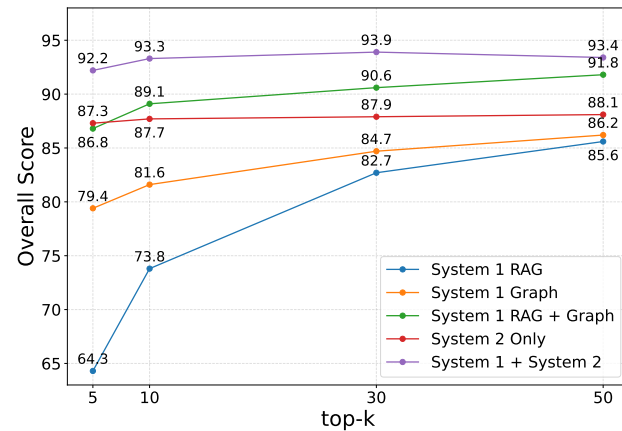


Figure 4: LoCoMo result across different top-k settings.

3.3.3. Effect of Backend Models

In this section, we analyze the effect of backend models (LLM, re-ranker, embedding) on Mnemis.

LLM. In Mnemis, LLM is responsible for memory component extraction, de-duplication and retrieval. Switching LLM from GPT-4o-mini to GPT-4.1-mini, Mnemis shows clear improvements across all datasets and question types, as show in tables 1 and 2. The same trend appears in the baseline methods, which suggests that the gains come from the stronger backend LLM rather than method specific factors. Given its favorable cost-performance trade-off, we recommend GPT-4.1-mini as the backend LLM for memory organization.

Re-ranker. Re-ranker organizes System-1 and System-2 search results to provide a compact context for the answer model. In the main experiments, we use Qwen3-Reranker-8B to obtain the best performance. We further evaluate Mnemis with two lightweight re-rankers: (1) Qwen3-Reranker-0.6B (Zhang et al., 2025) and (2) BGE-Reranker-V2-M3 (0.5B) (Chen et al., 2024). As shown in table 5, replacing the re-ranker with these smaller models results in only minor performance regressions.

Embedding Model. The embedding model contributes to the similarity based search across Mnemis. In our main experiments, we adopt Qwen3-Reranker-0.6B and reduce its embedding dimension from 1024 to 128 to

由于搜索查询在此路径中保持不变，它可能识别出一些关键点，但无法捕捉完整的时序事件序列。在实验中，约90.06% (1387 / 1540) 的查询获得了结果，因此我们报告这些有效查询的平均得分。结果符合我们的预期。结合两条路径后，所有类别均得到改善，这得益于它们互补的优势。

为了更直观，我们展示Mnemis在LoCoMo和LongMemEval-S基准测试中的成功案例，以证明引入系统2全局选择的有效性。如图3所示，当处理查询“Sam面临了何种健康问题，促使他改变生活方式？”时，基于相似性的搜索只能检索到“胃炎”，这仅仅是事件文本14-0中发现的表面原因。相比之下，配备了全局选择后，我们通过“身体健康” → “健康” → “健康事件” → “健康状况” → “胃炎”来定位表面原因，并通过“身体健康” → “健康” → “身体健康” → “健康因素” → “体重”来定位根本原因。在搜索路径中，中间类别（如“身体健康”和“健康因素”）自然地以摘要形式聚合了其后代节点的相关信息，从而进一步丰富了检索到的上下文。关于LongMemEval-S的案例，请参阅章节A。

3.3.2. top-k的影响

引入 top-k 参数是为了平衡答案成本和准确性。在主要实验中，我们使用 top-k = 10，这意味着生成最终答案时，上下文包含10个事件片段、20个实体和20条边。为了评估 top-k的影响，我们将其在5、10、30和50等值之间变化，并在与章节 3.3.1相同的配置下进行实验。

结果如图 4 所示，详细信息见表 10。将 top-k 从 10减少到5会导致大多数设置下的性能明显下降，这表明 top-k 为5不足以捕获用户查询所需的所有证据。系统-1 RAG 对这种减少尤其敏感，特别是在需要跨历史记录多个部分的多样化证据的多跳问题上。相比之下，检索信息密度更高的实体和边的系统-1图受影响较小。系统-1 RAG+ 图结合了事件片段、实体和边，表现出更稳定的性能，而在仅系统-2或系统-1 +2中应用重排序器进一步最小化了波动。

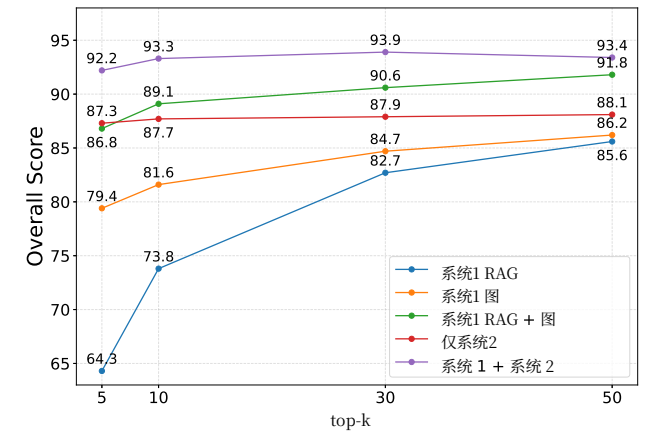


图 4: 不同 top-k 设置下的 LoCoMo 结果。

3.3.3. 后端模型的影响

在本节中，我们分析后端模型（大语言模型、重排序器、嵌入）对Mnemis的影响。

大语言模型。 在Mnemis中，大语言模型负责记忆组件提取、去重和检索。将后端大语言模型从GPT-4o-mini切换到GPT-4.1-mini后，Mnemis在所有数据集和问题类型上都显示出明显的改进，如表1和2所示。同样的趋势也出现在基线方法中，这表明性能提升源于更强大的后端大语言模型，而非方法特定的因素。鉴于其良好的性价比权衡，我们推荐使用GPT-4.1-mini作为记忆组织的后端大语言模型。

重排序器。 重排序器负责组织系统一和系统二的搜索结果，为答案模型提供紧凑的上下文。在主要实验中，我们使用Qwen3-Reranker-8B以获得最佳性能。我们进一步使用两种轻量级重排序器评估了Mnemis：(1) Qwen3-Reranker-0.6B (Zhang et al., 2025) 和 (2) BGE-Reranker-V2-M3 (0.5B) (Chen et al., 2024)。如表5所示，用这些更小的模型替换重排序器仅导致性能轻微下降。

嵌入模型。 嵌入模型负责在Mnemis中进行基于相似性的搜索。在我们的主要实验中，我们采用Qwen3重排序器-0.6B，并将其嵌入维度从1024降至128，以

Table 5: Detailed performance (LLM-as-a-Judge score) on LoCoMo by question type.

Re-ranker	Multi-Hop	Temporal	Open-Domain	Single-Hop	Overall
Qwen3-Reranker-0.6B	91.8	90.3	79.2	95.2	92.6
BGE-Reranker-V2-M3	90.1	90.0	77.1	96.4	92.7
Qwen3-Reranker-8B	91.8	90.3	82.3	96.2	93.3

Table 6: Detailed performance (LLM-as-a-Judge score) on LoCoMo by question type.

Embedder	Multi-Hop	Temporal	Open-Domain	Single-Hop	Overall
MiniLM	46.1	57.9	64.6	62.5	58.7
BGE-M3	50.0	74.1	63.5	74.0	69.0
Qwen3-Embedding-0.6B	64.9	76.6	67.7	76.5	73.8
Gemma-300M	62.4	78.8	60.4	82.9	76.9

control serving costs by using its MRL capability. To isolate and better understand the impact of embedding quality alone, we further evaluate System 1 RAG with three additional embedding models: (1) BGE-M3 (Chen et al., 2024) with dimension 1024, (2) all-MiniLM-L6-v2⁶ with dimension 384, and (3) Gemma-300M (Schechter Vera et al., 2025) with dimension 768. The results are presented in table 6.

4. Related Work

The core of AI memory lies in how they organize and retrieve past interactions. One straightforward way to organize LLMs’ memory is to treat them like individuals with hyperthymesia, supposing LLM can recall every past interaction without additional processing on historical messages. A line of work has therefore focused on enlarging the context window of LLMs (Liu et al., 2025; Peng et al., 2024). However, naively feeding the entire history can quickly become costly and inefficient in real-world applications due to the quadratic scaling of transformers with input length (Li et al., 2024; Wu et al., 2024b; Li et al., 2025b; Lin et al., 2025), and irrelevant information in historical messages may further dilute the context (Shi et al., 2023).

Another line of work borrows the idea of episodic memory (Tulving et al., 1972). It stores historical messages as Episodes and only retrieves relevant items when dealing with user queries, termed as retrieval-augmented generation (RAG) (Arslan et al., 2024; Lewis et al., 2020). Graph-RAG, incorporating concepts from semantic memory (Tulving et al., 1972), extracts *Entities* (key figures, objects, or concepts) and *Edges* (events or relationships connecting them) and organizes memory into a structured graph (Nan et al., 2025; Chhikara et al., 2025; Wang and Chen, 2025; Rasmussen et al., 2025). Some readers may note that GraphRAG (Edge et al., 2024) introduces a hierarchy concept similar to Mnemis. However, GraphRAG and Mnemis differ in two fundamental ways. First, GraphRAG constructs its hierarchy using community detection algorithms, where each lower-level node is assigned to a single parent. In contrast, Mnemis supports many-to-many mappings, allowing entities to belong to multiple higher-level categories and resulting in a more expressive and flexible hierarchy. Second, GraphRAG generates answers by independently querying each community and aggregating the outputs into a final response. In Mnemis, the model instead performs top-down hierarchical browsing to retrieve relevant memory components, and the final answer is produced based on the aggregated memory context rather than separate community-specific responses.

5. Conclusion

In this work, we introduce Mnemis, a unified memory framework to organize and retrieve AI memory. By combining a refined base graph for System-1 Similarity Search with a hierarchical graph designed to support System-2 Global Selection, Mnemis enables more accurate retrieval than existing RAG and Graph-RAG approaches on memory benchmarks, achieving 93.9 in LoCoMo and 91.6 on LongMemEval-S. While the results

表5：按问题类型划分的LoCoMo详细性能（LLM作为评判者的分数）。

重排序器	多跳	时序	开放域	单跳	总体
Qwen3重排序器-0.6B	91.8	90.3	79.2	95.2	92.6
BGE重排序器-V2-M3	90.1	90.0	77.1	96.4	92.7
Qwen3-Reranker-8B	91.8	90.3	82.3	96.2	93.3

表6：按问题类型划分的LoCoMo详细性能（LLM作为评判者的分数）。

嵌入器	多跳	时序	开放域	单跳	总体
MiniLM	46.1	57.9	64.6	62.5	58.7
BGE-M3	50.0	74.1	63.5	74.0	69.0
Qwen3-Embedding-0.6B	64.9	76.6	67.7	76.5	73.8
Gemma-300M	62.4	78.8	60.4	82.9	76.9

利用其MRL能力来控制服务成本。为了分离并更好地理解嵌入质量本身的影响，我们进一步使用三种额外的嵌入模型评估了系统1 RAG：(1) BGE-M3 (Chen等人, 2024)，维度1024；(2) all-MiniLM-L6-v2⁶，维度384；以及(3) Gemma-300M (Schechter Vera等人, 2025)，维度768。结果呈现在表6中。

4. 相关工作

AI记忆的核心在于如何组织和检索过去的交互。一种直接的组织大语言模型记忆的方式是将其视为患有超忆症的个体，假设大语言模型无需对历史消息进行额外处理就能回忆起每一次过去的交互。因此，一系列工作专注于扩大大语言模型的上下文窗口（Liu等人, 2025; Peng等人, 2024）。然而，在实际应用中，由于Transformer模型的复杂度随输入长度呈二次方增长（Li等人, 2024; Wu等人, 2024b; Li等人, 2025b; Lin等人, 2025），简单地输入整个历史记录会迅速变得成本高昂且效率低下，而历史消息中的不相关信息可能进一步稀释上下文（Shi等人, 2023）。

另一项工作借鉴了情景记忆（Tulving et al., 1972）的概念。它将历史消息存储为事件片段，仅在处理用户查询时检索相关项目，这被称为检索增强生成（Arslan et al., 2024; Lewis et al., 2020）。图检索增强生成则融合了语义记忆（Tulving et al., 1972）的概念，提取实体（关键人物、对象或概念）和边（连接它们的事件或关系），并将记忆组织成结构化图（Nan et al., 2025; Chhikara et al., 2025; Wang and Chen, 2025; Rasmussen et al., 2025）。一些读者可能注意到，GraphRAG（Edge et al., 2024）引入了与Mnemis相似的层级结构概念。然而，GraphRAG与Mnemis在两个方面存在根本差异。首先，GraphRAG使用社区检测算法构建其层级结构，其中每个低层级节点被分配给单一的父节点。相比之下，Mnemis支持多对多映射，允许实体属于多个高层级类别，从而形成更具表现力和灵活性的层级结构。其次，GraphRAG通过独立查询每个社区并将输出聚合为最终响应来生成答案。而在Mnemis中，模型执行的是自上而下的分层浏览来检索相关的记忆组件，最终答案是基于聚合的记忆上下文产生的，而非来自各自独立的社区特定响应。

5. 结论

在本工作中，我们介绍了Mnemis，一个用于组织和检索AI记忆的统一记忆框架。通过将用于系统一相似性搜索的精炼基础图与旨在支持系统2全局选择的层次图相结合，Mnemis在记忆基准测试上实现了比现有检索增强生成和图检索增强生成方法更准确的检索，在LoCoMo上达到93.9分，在LongMemEval-S上达到91.6分。尽管结果

⁶<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

⁶<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

are strong, several important directions remain open. In future work, we plan to support more data modalities and enhance global selection with more flexible graph traversal and planning mechanisms.

References

Muhammad Arslan, Hussam Ghanem, Saba Munawar, and Christophe Cruz. A survey on rag with llms. *Procedia computer science*, 246:3781–3790, 2024.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. <https://arxiv.org/abs/2504.19413>.

Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning, 2025. <https://arxiv.org/abs/2503.09516>.

Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011.

Sangyeop Kim, Yohan Lee, Sanghwa Kim, Hyunjong Kim, and Sungzoon Cho. Pre-storage reasoning for episodic memory: Shifting inference burden to memory for personalized dialogue. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 22096–22113, 2025.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai, Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan, Mingchuan Yang, Tong Xu, Wei Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang, Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. Memos: A memory os for ai system, 2025a. <https://arxiv.org/abs/2507.03724>.

Zhong-Zhi Li, Xiao Liang, Zihao Tang, Lei Ji, Peijie Wang, Haotian Xu, Xing W, Haizhen Huang, Weiwei Deng, Yeyun Gong, Zhijiang Guo, Xiao Liu, Fei Yin, and Cheng-Lin Liu. Tl;dr: Too long, do re-weighting for efficient llm reasoning compression, 2025b. <https://arxiv.org/abs/2506.02678>.

Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 881–893, 2024.

Xiao Liang, Zhongzhi Li, Yeyun Gong, Yelong Shen, Ying Nian Wu, Zhijiang Guo, and Weizhu Chen. Beyond pass@1: Self-play with variational problem synthesis sustains rlvr, 2025. <https://arxiv.org/abs/2508.14029>.

表现强劲，但仍有一些重要的方向有待探索。在未来的工作中，我们计划支持更多数据模态，并通过更灵活的图遍历和规划机制来增强全局选择。

参考文献

Muhammad Arslan, Hussam Ghanem, SabaMunawar, 以及 ChristopheCruz。A surveyon ragwith llms. *Procedia*计算机科学, 246:3781–3790, 2024.

Ali Behrouz, Peilin Zhong, 以及 Vahab Mirrokni. Titans: 测试时学习记忆. *arXiv*预印本 *arXiv:2501.00663*, 2024.

Jianlv Chen, ShitaoXiao, Peitian Zhang, Kun Luo, Defu Lian, 以及Zheng Liu。BGE M3嵌入模型：通过自知识蒸馏实现多语言、多功能、多粒度文本嵌入, 2024.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, 以及 Deshraj Yadav。Mem0: 构建具备可扩展长期记忆的生产就绪型AI智能体, 2025. <https://arxiv.org/abs/2504.19413>.

Gordon V Cormack、Charles LA Clarke 和 Stefan Buettcher。互逆排序融合优于孔多塞和个体排序学习方法。载于第32届国际*ACM SIGIR*信息检索研究与发展会议论文集，第758–759页，2009年。

Darren Edge、Ha Trinh、Newman Cheng、Joshua Bradley、Alex Chao、Apurva Mody、Steven Truitt、Dasha Metropolitansky、Robert Osazuwa Ness 和 Jonathan Larson。从局部到全局：一种用于查询聚焦式摘要的图RAG方法. *arXiv*预印本 *arXiv:2404.16130*，2024年。

Bowen Jin、Hansi Zeng、Zhenrui Yue、Jinsung Yoon、Sercan Arik、Dong Wang、Hamed Zamani 和 韩家炜。Search-R1：利用强化学习训练大语言模型进行推理与搜索引擎利用，2025年。<https://arxiv.org/abs/2503.09516>。

丹尼尔·卡尼曼。思考，快与慢。麦克米伦出版社，2011年。

Sangyeop Kim, Yohan Lee, Sanghwa Kim, Hyunjong Kim, 和 Sungzoon Cho。情景记忆的预存储推理：将推理负担转移至记忆以实现个性化对话。载于计算语言学协会研究结果： *EMNLP 2025*，第22096–22113页，2025。

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, 等。面向知识密集型NLP任务的检索增强生成。神经信息处理系统进展，33:9459–9474，2020。

李志宇，席晨阳，李春雨，陈鼎，陈博宇，宋世超，牛思敏，王瀚宇，杨佳伟，唐晨，余庆辰，赵继豪，王晔昭辉，刘鹏，林泽豪，王鹏远，霍嘉豪，陈天翼，陈凯，李可航，陶真，赖华毅，吴昊，唐博，王正仁，范兆鑫，张宁豫，张林峰，严骏驰，杨明川，许童，许伟，陈华钧，王昊奋，杨洪康，张文涛，许志钦，陈思衡，和熊飞宇。Memos：面向人工智能系统的内存操作系统，2025a。<https://arxiv.org/abs/2507.03724>。

李忠智、梁潇、Zihao Tang、纪磊、王培杰、徐昊天、W Xing、黄海珍、邓伟伟、宫业云、郭志江、刘潇、殷飞和刘成林。太长不看：太长，进行重加权以实现高效的大语言模型推理压缩，2025b。<https://arxiv.org/abs/2506.02678>。

李卓婉、李成、张明阳、梅俏竹和Michael Bendersky。检索增强生成还是长上下文大语言模型？一项综合研究及混合方法。载于2024年自然语言处理经验方法会议论文集：工业界轨道，第881–893页，2024年。

梁潇、李忠智、宫业云、沈业龙、吴英年、郭志江、陈为铸。超越Pass@1：通过变分问题合成进行自我博弈以维持RLVR，2025。<https://arxiv.org/abs/2508.14029>。

Zhenghao Lin, Zihao Tang, Xiao Liu, Yeyun Gong, Yi Cheng, Qi Chen, Hang Li, Ying Xin, Ziyue Yang, Kailai Yang, Yu Yan, Xiao Liang, Shuai Lu, Yiming Huang, Zheheng Luo, Lei Qu, Xuan Feng, Yaoxiang Wang, Yuqing Xia, Feiyang Chen, Yuting Jiang, Yasen Hu, Hao Ni, Binyang Li, Guoshuai Zhao, Jui-Hao Chiang, Zhongxin Guo, Chen Lin, Kun Kuang, Wenjie Li, Yelong Shen, Jian Jiao, Peng Cheng, and Mao Yang. Sigma: Differential rescaling of query, key and value for efficient language models, 2025. <https://arxiv.org/abs/2501.13629>.

Jiaheng Liu, Dawei Zhu, Zhiqi Bai, Yancheng He, Huanxuan Liao, Haoran Que, Zekun Wang, Chenchen Zhang, Ge Zhang, Jiebin Zhang, Yuanxing Zhang, Zhuo Chen, Hangyu Guo, Shilong Li, Ziqiang Liu, Yong Shan, Yifan Song, Jiayi Tian, Wenhao Wu, Zhejian Zhou, Ruijie Zhu, Junlan Feng, Yang Gao, Shizhu He, Zhoujun Li, Tianyu Liu, Fanyu Meng, Wenbo Su, Yingshui Tan, Zili Wang, Jian Yang, Wei Ye, Bo Zheng, Wangchunshu Zhou, Wenhao Huang, Sujian Li, and Zhaoxiang Zhang. A comprehensive survey on long context language modeling, 2025. <https://arxiv.org/abs/2503.17407>.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.

Jaayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science, 2025. <https://arxiv.org/abs/2508.03341>.

Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. Secom: On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations*, 2025. <https://openreview.net/forum?id=xKDZAW0He3>.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=wHBfxhZu1u>.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. <https://arxiv.org/abs/2501.13956>.

Henrique* Schechter Vera, Sahil* Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiyl Wang, Zhe Li, Gus Martins, Jinhyuk Lee, Mark Sherwood, Juyeong Ji, Renjie Wu, Jingxiao Zheng, Jyotinder Singh, Abheesht Sharma, Divya Sreepat, Aashi Jain, Adham Elarabawy, AJ Co, Andreas Doumanoglou, Babak Samari, Ben Hora, Brian Potetz, Dahun Kim, Enrique Alfonseca, Fedor Moiseev, Feng Han, Frank Palma Gomez, Gustavo Hernández Ábrego, Hesén Zhang, Hui Hui, Jay Han, Karan Gill, Ke Chen, Koert Chen, Madhuri Shanbhogue, Michael Boratko, Paul Suganthan, Sai Meher Karthik Duddu, Sandeep Mariserla, Setareh Ariafer, Shanfeng Zhang, Shijie Zhang, Simon Baumgartner, Sonam Goenka, Steve Qiu, Tanmaya Dabral, Trevor Walker, Vikram Rao, Waleed Khawaja, Wenlei Zhou, Xiaoqi Ren, Ye Xia, Yichang Chen, Yi-Ting Chen, Zhe Dong, Zhongli Ding, Francesco Visin, Gaël Liu, Jiageng Zhang, Kathleen Kenealy, Michelle Casbon, Ravin Kumar, Thomas Mesnard, Zach Gleicher, Cormac Brick, Olivier Lacombe, Adam Roberts, Yunhsuan Sung, Raphael Hoffmann, Tris Warkentin, Armand Joulin, Tom Duerig, and Mojtaba Seyedhosseini. Embeddinggemma: Powerful and lightweight text representations. 2025. <https://arxiv.org/abs/2509.20354>.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pages 31210–31227. PMLR, 2023.

Zihao Tang, Zheqi Lv, Shengyu Zhang, Fei Wu, and Kun Kuang. Modelgpt: Unleashing llm’s capabilities for tailored model generation, 2024. <https://arxiv.org/abs/2402.12408>.

林正浩、Zihao Tang、刘潇、宫业云、程毅、陈琦、李航、辛颖、杨子悦、杨凯来、严宇、梁潇、陆帅、黄一鸣、罗哲恒、曲磊、冯璇、王耀祥、夏雨晴、Feiyang Chen、蒋雨婷、胡亚森、倪浩、李彬阳、赵国帅、蒋睿豪、郭忠鑫、林晨、邝琨、李文杰、沈业龙、焦剑、程鹏、杨旻。Sigma: 对查询、键和值进行差分重缩放以实现高效语言模型，2025。 <https://arxiv.org/abs/2501.13629>。

刘嘉恒、朱大伟、白志奇、何彦成、廖焕轩、阙浩然、王泽坤、张晨晨、张戈、张杰斌、张元星、陈卓、郭航宇、李士龙、刘自强、单勇、宋一凡、田佳怡、吴文浩、周哲健、朱瑞杰、冯俊兰、高阳、何世柱、李周峻、刘天宇、孟繁宇、苏文博、谭映水、王子立、杨健、叶伟、郑波、周王春澍、黄文浩、李素建、张兆翔。长上下文语言建模全面综述，2025。 <https://arxiv.org/abs/2503.17407>。

Adyasha Maharana、李东浩、Sergey Tulyakov、Mohit Bansal、Francesco Barbieri 和方宇威。评估大语言模型智能体的超长期对话记忆。 *arXiv* 预印本 *arXiv:2402.17753*，2024。

南佳岩、马文全、吴文龙和陈一泽。Nemori：受认知科学启发的自组织智能体记忆，2025。 <https://arxiv.org/abs/2508.03341>。

欧阳思如、严俊、I Hsu、陈彦飞、姜珂、王子丰、韩汝军、Long T Le、Samira Daruki、唐相如等。Reasoningbank：利用推理记忆实现规模化智能体自进化。 *arXiv* 预印本 *arXiv:2509.25140*，2025。

潘卓石、吴千慧、江慧强、罗旭芳、程浩、李东升、杨雨晴、林智仁、赵慧琦、邱莉莉和高剑峰。Secom: 论个性化对话智能体的记忆构建与检索。收录于第十三届国际学习表征会议，2025。 <https://openreview.net/forum?id=xKDZAW0He3>。

Bowen Peng、Jeffrey Quesnelle、Honglu Fan 和 Enrico Shippole。YaRN：大语言模型的高效上下文窗口扩展。发表于第十二届国际学习表征会议，2024年。 <https://openreview.net/forum?id=wHBfxhZu1u>。

Preston Rasmussen、Pavlo Paliychuk、Travis Beauvais、Jack Ryan 和 Daniel Chalef。Zep：一种用于智能体记忆的时序知识图谱架构，2025年。 <https://arxiv.org/abs/2501.13956>。

Henrique* Schechter Vera, Sahil* Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiyl Wang, Zhe Li, Gus Martins, Jinhyuk Lee, Mark Sherwood, Juyeong Ji, Renjie Wu, Jingxiao Zheng, Jyotinder Singh, Abheesht Sharma, Divya Sreepat, Aashi Jain, Adham Elarabawy, AJ Co, Andreas Doumanoglou, Babak Samari, Ben Hora, Brian Potetz, Dahun Kim, Enrique Alfonseca, Fedor Moiseev, Feng Han, Frank Palma Gomez, Gustavo Hernández Ábrego, Hesén Zhang, Hui Hui, Jay Han, Karan Gill, Ke Chen, Koert Chen, Madhuri Shanbhogue, Michael Boratko, Paul Suganthan, Sai Meher Karthik Duddu, Sandeep Mariserla, Setareh Ariafer, Shanfeng Zhang, Shijie Zhang, Simon Baumgartner, Sonam Goenka, Steve Qiu, Tanmaya Dabral, Trevor Walker, Vikram Rao, Waleed Khawaja, Wenlei Zhou, Xiaoqi Ren, Ye Xia, Yichang Chen, Yi-Ting Chen, Zhe Dong, Zhongli Ding, Francesco Visin, Gaël Liu, Jiageng Zhang, Kathleen Kenealy, Michelle Casbon, Ravin Kumar, Thomas Mesnard, Zach Gleicher, Cormac Brick, Olivier Lacombe, Adam Roberts, Yunhsuan Sung, Raphael Hoffmann, Tris Warkentin, Armand Joulin, Tom Duerig 和 Mojtaba Seyedhosseini. Embeddinggemma: 强大且轻量级的文本表示。2025年。 <https://arxiv.org/abs/2509.20354>。

Freda Shi、Xinyun Chen、Kanishka Misra、Nathan Scales、David Dohan、Ed H Chi、Nathanael Schärli 和 Denny Zhou。大语言模型容易被无关上下文分散注意力。发表于 国际机器学习大会，第 31210–31227 页。PMLR, 2023。

Zihao Tang、Zheqi Lv、Shengyu Zhang、Fei Wu 和 Kun Kuang。ModelGPT：释放大语言模型能力，实现定制化模型生成，2024。 <https://arxiv.org/abs/2402.12408>。

Endel Tulving et al. Episodic and semantic memory. *Organization of memory*, 1(381-403):1, 1972.

Liang Wang, Haonan Chen, Nan Yang, Xiaolong Huang, Zhicheng Dou, and Furu Wei. Chain-of-retrieval augmented generation. In *NeurIPS 2025*, October 2025. <https://www.microsoft.com/en-us/research/publication/chain-of-retrieval-augmented-generation/>.

Yu Wang and Xi Chen. Mirix: Multi-agent memory system for llm-based agents, 2025. <https://arxiv.org/abs/2507.07957>.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. 2024a. <https://arxiv.org/abs/2410.10813>.

Tao Wu, Mengze Li, Jingyuan Chen, Wei Ji, Wang Lin, Jinyang Gao, Kun Kuang, Zhou Zhao, and Fei Wu. Semantic alignment for multimodal large language models. In Jianfei Cai, Mohan S. Kankanhalli, Balakrishnan Prabhakaran, Susanne Boll, Ramanathan Subramanian, Liang Zheng, Vivek K. Singh, Pablo César, Lexing Xie, and Dong Xu, editors, *Proceedings of the 32nd ACM International Conference on Multimedia, MM 2024, Melbourne, VIC, Australia, 28 October 2024 - 1 November 2024*, pages 3489–3498. ACM, 2024b. doi: 10.1145/3664647.3681014. <https://doi.org/10.1145/3664647.3681014>.

Tao Wu, Jingyuan Chen, Wang Lin, Mengze Li, Yumeng Zhu, Ang Li, Kun Kuang, and Fei Wu. Embracing imperfection: Simulating students with diverse cognitive levels using llm-based agents. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 9887–9908. Association for Computational Linguistics, 2025a. <https://aclanthology.org/2025.acl-long.488/>.

Tao Wu, Jingyuan Chen, Wang Lin, Jian Zhan, Mengze Li, Kun Kuang, and Fei Wu. Personalized distractor generation via mcts-guided reasoning reconstruction. *CoRR*, abs/2508.11184, 2025b. doi: 10.48550/ARXIV.2508.11184. <https://doi.org/10.48550/arXiv.2508.11184>.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.

Sizhe Zhou. A simple yet strong baseline for long-term conversational memory of llm agents, 2025. <https://arxiv.org/abs/2511.17208>.

Endel Tulvin g 等人. 情景记忆与语义记忆. 记忆的组织, 1(381-403):1, 1972。

王亮、陈浩楠、杨楠、黄晓龙、窦志成和韦福如. 检索链增强生成. 发表于*NeurIPS 2025*, 2025年10月。
<https://www.microsoft.com/en-us/research/publication/chain-of-retrieval-augmented-generation/>。

王宇和陈曦. Mirix: 面向基于大语言模型的智能体的多智能体记忆系统, 2025. <https://arxiv.org/abs/2507.07957>。

吴迪、王宏伟、余文浩、张宇伟、Kai-WeiChang和俞栋. Longmemeval: 聊天助手长期交互记忆基准测试. 2024a. <https://arxiv.org/abs/2410.10813>。

吴涛、李梦泽、陈景元、纪伟、林旺、高金阳、邝琨、赵周和Fei Wu. 多模态大语言模型的语义对齐. 蔡剑飞、Mohan S. Kankanhalli、Balakrishnan Prabhakaran、Susanne Boll、Ramanathan Subramanian、郑良、Vivek K. Singh、Pablo César、谢乐欣和徐东编辑, 第32届ACM国际多媒体会议论文集, *MM 2024*, 澳大利亚维多利亚州墨尔本, 2024年10月28日 - 2024年11月1日, 第3489–3498页. ACM, 2024b. doi: 10.1145/3664647.3681014. <https://doi.org/10.1145/3664647.3681014>。

吴涛、陈景元、林旺、李梦泽、朱雨萌、李昂、邝琨和Fei Wu. 拥抱不完美: 使用基于大语言模型的智能体模拟具有多样认知水平的学生. 车万翔、Joyce Nabende、Ekaterina Shutova和Mohammad Taher Pilehvar编辑, 计算语言学协会第63届年会论文集 (第一卷: 长论文), *ACL 2025*, 奥地利维也纳, 2025年7月27日 - 2025年8月1日, 第9887–9908页. 计算语言学协会, 2025a. <https://aclanthology.org/2025.acl-long.488/>。

吴涛、陈景元、林旺、詹健、李梦泽、邝琨、吴飞. 通过MCTS引导的推理重建实现个性化干扰项生成. *CoRR*, abs/2508.11184, 2025b. doi: 10.48550/ARXIV.2508.11184. <https://doi.org/10.48550/arXiv.2508.11184>。

Sikuan Yan、Xiufeng Yang、Zuchao Huang、Ercong Nie、Zifeng Ding、Zonggen Li、Xiaowen Ma、Kristian Kersting、Jeff Z Pan、Hinrich Schütze等. Memory-r1: 通过强化学习增强大语言模型智能体以管理和利用记忆. *arXiv预印本arXiv:2508.19828*, 2025。

Yanzhao Zhang、Mingxin Li、Dingkun Long、Xin Zhang、Huan Lin、Baosong Yang、Pengjun Xie、An Yang、Dayiheng Liu、Junyang Lin、Fei Huang、Jingren Zhou. Qwen3嵌入模型: 通过基础模型推进文本嵌入和重排序. *arXiv预印本arXiv:2506.05176*, 2025。

Sizhe Zhou. 一个用于大语言模型智能体长期对话记忆的简单而强大的基线, 2025. <https://arxiv.org/abs/2511.17208>。

A. Case Study

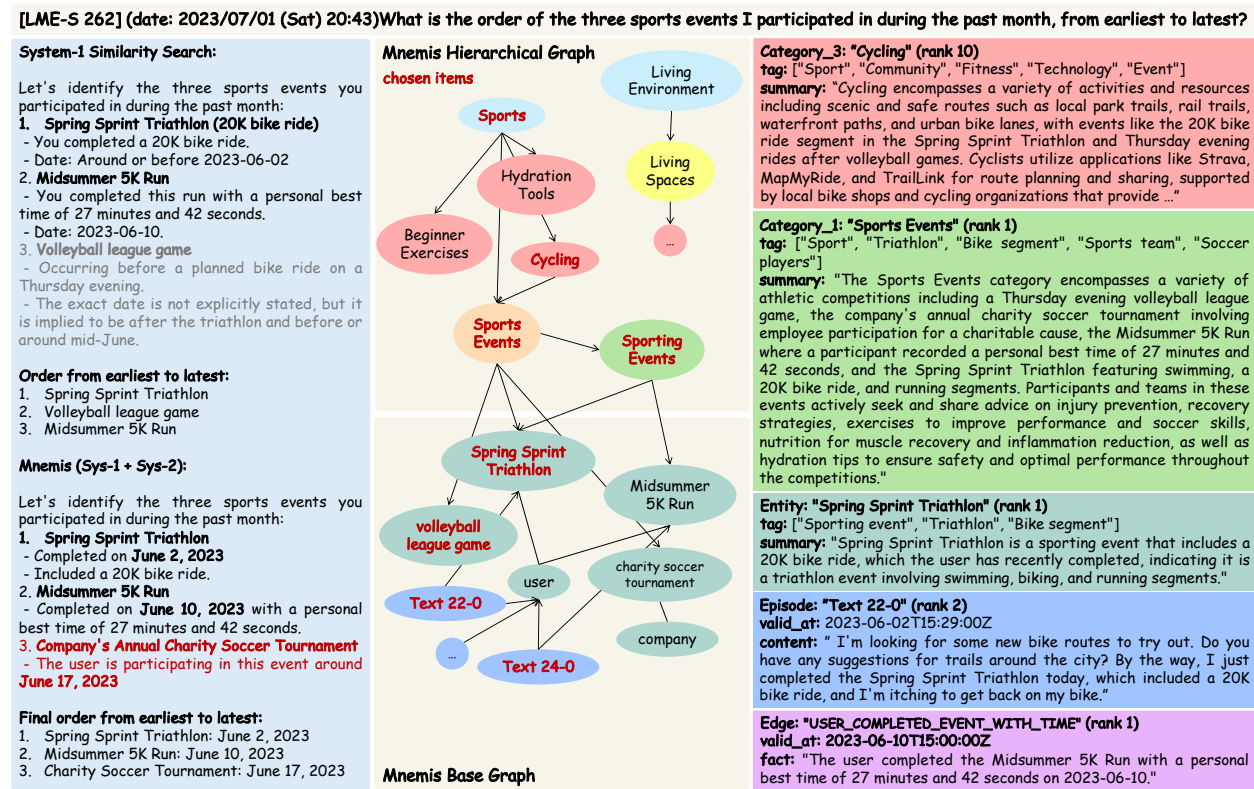


Figure 5: Mnemis win case on LongMemEval-S benchmark, where Mnemis successfully retrieve "Company's Annual Charity Soccer Tournament" from "Sports Events".

fig. 5 provides another win case of Mnemis on LongMemEval-S. This question asks the LLM to order the three sports events the user participated in during June 2026. Similarity-based retrieval struggles here, as a limited top-k often fails to surface all relevant sports events. In contrast, Mnemis can simply start from top-level category "Sports" and browse down through Category "Sports Events" and "Sporting Events" and then reliably retrieve all events.

Although the ground truth entities "Midsummer 5K Run" and "charity soccer tournament" are filtered out in the final answer context due to limited top-k, their related edges, such as "The user participates in the company's annual charity soccer tournament. (2023-06-17 - now)" and "The user completed a 5K run at the Midsummer 5K Run with a personal best time of 27 minutes and 42 seconds. (2023-06-10 - now)" are still retrieved with rank 9 and 2 respectively. Besides, category nodes like "Sports Events", summarizing the content of their children nodes, also provide sufficient content for the model to answer the question correctly.

B. Further Benchmark Results

B.1. Evaluation Metrics

In the main experiments, we adopt LLM-as-a-Judge (LAJ) as the primary evaluation metric for comparing Mnemis against all baselines, following common practice. We directly use the official judging prompt to ensure comparability. To further assess the reliability of LAJ, we conduct a small-scale human evaluation.

Specifically, we manually sample Mnemis's outputs on LoCoMo, inspecting both win and loss cases (three labeled correct and three labeled incorrect per user), resulting in 60 questions sampled from 1,540 evaluation instances, as shown in table 7. Human evaluation is performed using the same grading criteria as LoCoMo.

A. 案例研究

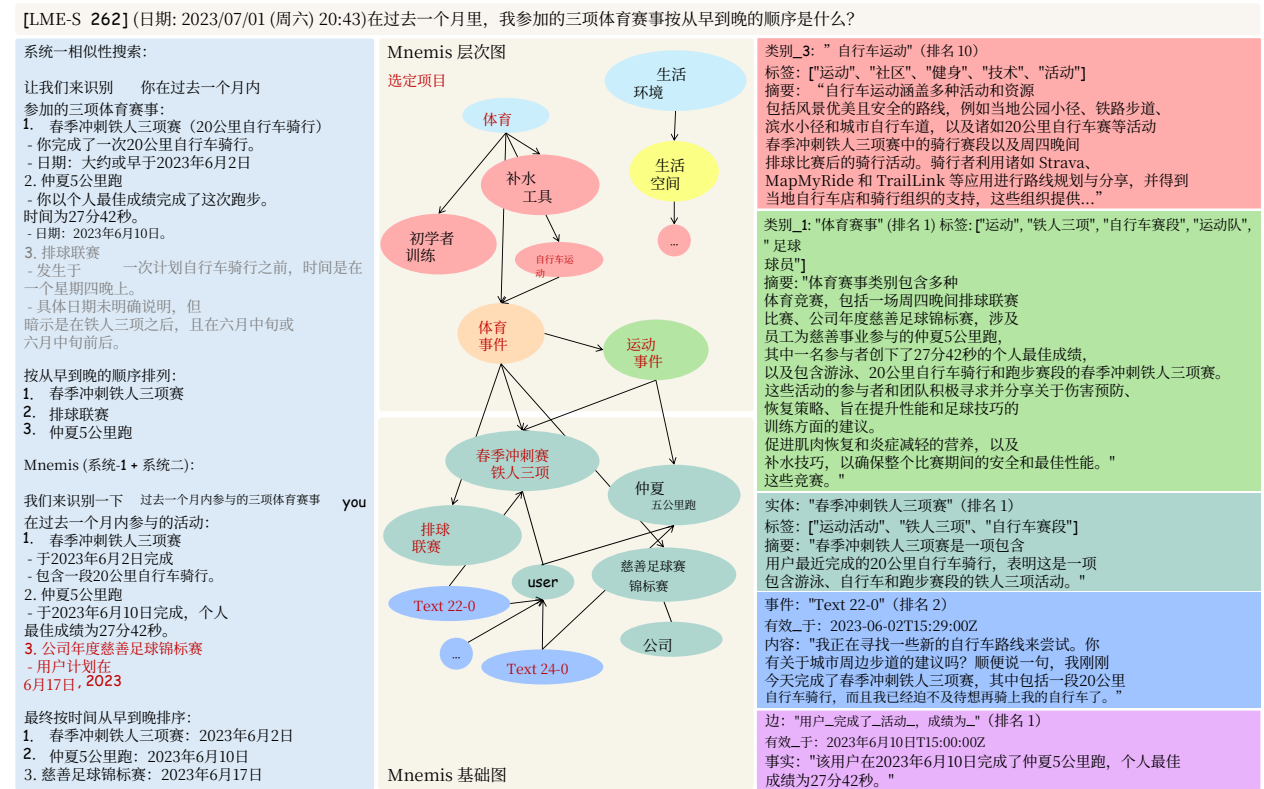


图 5: Mnemis 在 LongMemEval-S 基准上的成功案例, 其中 Mnemis 成功从“体育赛事”中检索出“公司年度慈善足球锦标赛”。

图 5 提供了 Mnemis 在 LongMemEval-S 上的另一个成功案例。这个问题要求大语言模型对用户 2026 年 6 月参加的三项体育赛事进行排序。相似性检索在此处遇到困难, 因为有限的 top-k 通常无法召回所有相关的体育赛事。相比之下, Mnemis 可以简单地从顶级类别“体育”开始, 向下浏览类别“体育赛事”和“运动赛事”, 然后可靠地检索出所有事件。

尽管真实情况实体“仲夏5公里跑”和“慈善足球锦标赛”因有限的 top-k 而被过滤在最终答案上下文之外, 但其相关的边, 例如“用户参加了公司年度慈善足球锦标赛。(2023年6月17日 - 现在)”和“用户在仲夏5公里跑中以27分42秒的个人最佳成绩完成了五公里跑。(2023年6月10日 - 现在)”, 仍分别以第9和第2的排名被检索到。此外, 像“体育赛事”这样的类别节点, 总结了其子节点的内容, 也为模型正确回答问题提供了足够的内容。

B. 进一步的基准测试结果

B.1. 评估指标

在主实验中, 我们遵循常见做法, 采用大语言模型作为评判者 (LAJ) 作为主要评估指标, 以比较 Mnemis 与所有基线方法。我们直接使用官方评判提示以确保可比性。为了进一步评估 LAJ 的可靠性, 我们进行了小规模的人工评估。

具体而言, 我们手动采样了 Mnemis 在 LoCoMo 上的输出, 检查了获胜和失败案例 (每位用户标注三个正确和三个错误), 从 1,540 个评估实例中抽取了 60 个问题, 如表 7 所示。人工评估采用与 LoCoMo 相同的评分标准进行。

Table 7: Sampled Question ID for Human Evaluation in LoCoMo.

User IDs	0	1	2	3	4
LLM-as-a-Judge True Question IDs	31, 8, 74	13, 74, 11	7, 26, 60	6, 160, 59	124, 161, 80
LLM-as-a-Judge False Question IDs	7, 138, 137	73, 47, 20	17, 49, 63	195, 92, 53	165, 166, 6
User IDs	5	6	7	8	9
LLM-as-a-Judge True Question IDs	104, 110, 23	47, 65, 96	25, 97, 94	126, 147, 32	99, 156, 54
LLM-as-a-Judge False Question IDs	55, 19, 119	1, 62, 31	117, 77, 23	102, 39, 70	7, 1, 61

Table 8: Detailed performance (LLM-as-a-Judge score) on LoCoMo by question type. Following the common practice, Category 5 (Adversarial) is excluded from the results.

Methods	Multi-Hop	Temporal	Open-Domain	Single-Hop	Overall
#Questions	282	321	96	841	1540
Full Context	77.2	74.2	56.6	86.9	80.6
RAG	64.9	76.6	67.7	76.5	73.8
LangMem	71.0	50.8	59.0	84.5	73.4
Mem0	68.2	56.9	47.9	71.4	66.3
Zep	53.7	60.2	43.8	66.9	61.6
Nemori	75.1	77.6	51.0	84.9	79.5
PREMem	61.0	74.8	46.9	66.2	65.8
EMem-G	79.6	80.8	71.7	90.5	85.3
MIRIX	83.7	88.4	65.6	85.1	85.4
EverMemOS	91.1	89.7	70.8	96.1	92.3
MemU	88.3	92.5	77.1	94.9	92.1
Mnemis	92.9	90.7	79.2	97.1	93.9

Across these samples, we did not observe any misalignment between the LLM judge and human evaluation. We include the detailed judging results in our GitHub repository.

We also note several strong recently proposed baselines: MIRIX (Wang and Chen, 2025), MemU⁷, Emergence-Mem⁸. However, due to incomplete implementation details, such as backend model configuration or system workflow, we could not obtain their results under comparable settings. To maintain fairness, we therefore exclude them from the main evaluation. Here, we report their best publicly available performance, regardless of configuration differences, to provide readers with a broader view of the current landscape.

The full results are reported in tables 8 and 9 and align with the findings summarized in section 3.2. Mnemis consistently outperforms all baseline methods across both benchmarks.

C. Detailed Performance Impact of top-*k*

Beyond the overall trend as shown in fig. 4, a closer breakdown by question type in table 10 reveals distinct behavioral patterns across retrieval strategies. For System-1 RAG, increasing top-*k* consistently improves performance across all categories, with particularly large gains on Multi-Hop subset (49.6→81.6). This suggests that the retrieved text contains necessary but scattered evidence, and restricting retrieval too aggressively leads to missing critical evidence. However, even at high top-*k*, RAG remains relatively weak on Multi-Hop (peaking at 81.6), indicating difficulty in identifying temporally aligned evidence without explicit structure.

In contrast, System-1 Graph shows a stronger starting point, especially on structured attributes (e.g., Single-Hop: 86.7 with top-*k* = 5), meaning the graph format inherently surfaces salient relational information

表 7: LoCoMo 中用于人工评估的抽样问题ID。

用户ID	0	1	2	3	4
大语言模型作为裁判 真实问题ID	31, 8, 74	13, 74, 11	7, 26, 60	6, 160, 59	124, 161, 80
大语言模型作为裁判 错误 问题ID	7, 138, 137	73, 47, 20	17, 49, 63	195, 92, 53	165, 166, 6
用户ID	5	6	7	8	9
大语言模型作为裁判 真实 问题ID	104, 110, 23	47, 65, 96	25, 97, 94	126, 147, 32	99, 156, 54
大语言模型作为裁判 错误 问题ID	55, 19, 119	1, 62, 31	117, 77, 23	102, 39, 70	7, 1, 61

表8: 按问题类型划分的LoCoMo详细性能（LLM作为评判者的分数）。遵循惯例，类别5（对抗性）已从结果中排除。

方法	多跳	时序	开放域	单跳	总体
#问题	282	321	96	841	1540
完整上下文	77.2	74.2	56.6	86.9	80.6
RAG	64.9	76.6	67.7	76.5	73.8
LangMem	71.0	50.8	59.0	84.5	73.4
Mem0	68.2	56.9	47.9	71.4	66.3
Zep	53.7	60.2	43.8	66.9	61.6
Nemori	75.1	77.6	51.0	84.9	79.5
PREMem	61.0	74.8	46.9	66.2	65.8
EMem-G	79.6	80.8	71.7	90.5	85.3
MIRIX	83.7	88.4	65.6	85.1	85.4
EverMemOS	91.1	89.7	70.8	96.1	92.3
MemU	88.3	92.5	77.1	94.9	92.1
Mnemis	92.9	90.7	79.2	97.1	93.9

在这些样本中，我们未观察到LLM评判器与人工评估之间存在任何不一致。我们已将详细的评判结果包含在我们的GitHub仓库中。

我们也注意到几个近期提出的强大基线方法：MIRIX (Wang and Chen,2025)、MemU⁷, Emergence-Mem⁸。然而，由于实现细节不完整，例如后端模型配置或系统工作流，我们无法在可比较的设置下获得它们的结果。为了保持公平性，我们因此将它们排除在主评估之外。在此，我们报告其最佳的公开可用性能，不论配置差异如何，以便为读者提供当前格局的更广阔视野。

完整结果报告于表8和9中，并与第3.2节总结的发现一致。Mnemis 在两个基准测试中均持续优于所有基线方法。

C. top-*k* 的详细性能影响

除了如图4所示的总体趋势外，表10中按问题类型的细分揭示了不同检索策略的独特行为模式。对于系统-1 RAG，增加 top-*k* 值持续提升所有类别的性能，尤其是在多跳子集上提升显著（49.6→81.6）。这表明检索到的文本包含必要但分散的证据，过于激进地限制检索会导致遗漏关键证据。然而，即使在较高的 top-*k* 值下，RAG 在多跳问题上仍然相对较弱（峰值 81.6），这表明在没有显式结构的情况下，识别时序对齐的证据存在困难。

相比之下，系统-1 图展现出一个更强的起点，尤其是在结构化属性上（例如，Single-Hop: 86.7, top-*k* = 5），这意味着图格式本质上就能凸显出显著的关系信息。

⁷<https://github.com/NevaMind-AI/memU>

⁸<https://www.emergence.ai/blog/sota-on-longmemeval-with-rag>

⁷<https://github.com/NevaMind-AI/memU>

⁸<https://www.emergence.ai/blog/sota-on-longmemeval-with-rag>

Table 9: Detailed performance (LLM-as-a-Judge score) on LongMemEval-S, categorized by question type: single-session-user (SSU), multi-session (MS), single-session-preference (SSP), temporal reasoning (TR), knowledge update (KU), and single-session-assistant (SSA).

Methods	SSU	MS	SSP	TR	KU	SSA	Overall
#Questions	70	133	30	133	78	56	500
Full Context	85.7	51.1	16.7	60.2	76.9	98.2	65.6
RAG	82.9	54.9	86.7	67.7	80.8	94.6	72.6
PREMem	92.9	57.1	36.7	59.4	84.6	12.5	60.8
Mem0	94.3	66.9	86.7	75.9	87.2	96.4	80.8
Nemori	90.0	55.6	86.7	72.2	79.5	92.9	74.6
EverMemOS	100.0	78.5	96.7	71.2	87.2	78.6	82.0
EMem-G	94.8	82.6	50.0	83.7	94.4	87.5	84.9
EmergenceMem	98.6	81.2	60.0	85.7	83.3	100.0	86.0
Mnemis	98.6	86.5	100.0	86.5	93.6	100.0	91.6

without requiring large retrieval volumes. Yet, the improvement curve is flatter compared to RAG, especially for Temporal and Open-Domain questions, where explicit structural relations help but cannot fully compensate for missing richer semantic context.

When combining both storage types in System-1 RAG + Graph, the benefits become additive: performance improves steadily and remains stable even under lower top- k settings. Notably, Multi-Hop and Temporal queries benefit the most from this hybrid storage design (e.g., 83.2 vs. 61.4/70.1 at top- $k = 5$), demonstrating that structured and unstructured information provide complementary retrieval signals.

Finally, applying System-2 and re-ranker, either alone or on top of System-1, further suppresses top- k sensitivity. System-1 + System-2 consistently delivers the highest and most stable performance (92.2–93.9 Overall), showing that even when overly large or overly sparse retrieval occurs, the re-ranker filters noise and prioritizes the most relevant evidence.

D. Prompts

To implement our results, we release the key prompts in our procedure. Below is the instruction to build hierarchical graph.

```
1 def extract_category_nodes(context: dict[str, Any], layer: int, prev_example: str) -> list
  [Message]:
2     sys_prompt = f"""You are an AI assistant specialized in semantic categorization of
      nodes.
3 # INSTRUCTIONS:
4
5 You are given a list of node names, each prefixed with an index, each followed with a
  brief description of the name (e.g., 1. dog: [domestic animal]).
6 Your task is to:
7
8 1. Group the nodes into semantically meaningful categories based on shared attributes,
  considering both inherent characteristics of the node names and the DESCRIPTIONS of the
  nodes, NOT relying solely on the DESCRIPTIONS.
9     All EXISTING CATEGORIES are provided for you.
10
11     - If a node's attribute matches an existing category, it should be added under that
    category.
12     - If a node name has attributes that do not match any existing category, create a new
    category and add it.
13     - The category name MUST NOT include the word "and" as a connector.
```

表 9: LongMemEval-S 上的详细性能（LLM作为评判者的分数），按问题类型分类：单会话用户（SSU）、多会话（MS）、单会话偏好（SSP）、时序推理（TR）、知识更新（KU）和单会话助手（SSA）。

方法	SSU	MS	SSP	TR	KU	SSA	总体
问题	70	133	30	133	78	56	500
完整上下文	85.7	51.1	16.7	60.2	76.9	98.2	65.6
RAG	82.9	54.9	86.7	67.7	80.8	94.6	72.6
PREMem	92.9	57.1	36.7	59.4	84.6	12.5	60.8
Mem0	94.3	66.9	86.7	75.9	87.2	96.4	80.8
Nemori	90.0	55.6	86.7	72.2	79.5	92.9	74.6
EverMemOS	100.0	78.5	96.7	71.2	87.2	78.6	82.0
EMem-G	94.8	82.6	50.0	83.7	94.4	87.5	84.9
EmergenceMem	98.6	81.2	60.0	85.7	83.3	100.0	86.0
Mnemis	98.6	86.5	100.0	86.5	93.6	100.0	91.6

且无需大量检索。然而，其提升曲线相较于检索增强生成更为平缓，尤其是在时序和开放域问题上，显式的结构关系虽有帮助，但无法完全弥补缺失的更丰富语义上下文。

当在系统-1 RAG + 图中结合两种存储类型时，其优势是叠加的：性能稳步提升，即使在较低 top- k 设置下也能保持稳定。值得注意的是，多跳和时序查询从这种混合存储设计中受益最大（例如，在 top- $k = 5$ 时，83.2 对比 61.4/70.1），这表明结构化和非结构化信息提供了互补的检索信号。

最终，应用系统二和重排序器，无论是单独使用还是叠加在系统一之上，都能进一步抑制 top- k 敏感性。系统-1 + 系统二始终提供最高且最稳定的性能（总体 92.2–93.9），这表明即使发生过大规模或过于稀疏的检索，重排序器也能过滤噪声并优先处理最相关的证据。

D. 提示词

为实现我们的结果，我们发布了流程中的关键提示词。以下是构建层次图的指令。

```
1 def extract_category_nodes(context: dict[str, Any], layer: int, prev_example: str) -> list
[Message]:
2     node
    ea
6     y
8     i
    NO
11
catl
create a new category and add it.
13     - The category name MUST NOT include the word "and" as a connector.
```


Table 10: Detailed performance (LLM-as-a-Judge score) on LoCoMo by question type.

Settings	top- <i>k</i>	Multi-Hop	Temporal	Open-Domain	Single-Hop	Overall
System-1 RAG	5	49.6	70.1	65.6	66.8	64.3
	10	64.9	76.6	67.7	76.5	73.8
	30	77.3	82.9	69.8	86.0	82.7
	50	81.6	84.1	71.9	89.1	85.6
System-1 Graph	5	79.4	61.4	75.0	86.7	79.4
	10	84.8	62.6	74.0	88.6	81.6
	30	87.6	66.4	77.1	91.6	84.7
	50	90.1	68.5	79.2	92.4	86.2
System-1 RAG + Graph	5	81.6	83.2	71.9	91.6	86.8
	10	85.1	84.7	75.0	93.7	89.1
	30	87.9	86.3	76.0	94.9	90.6
	50	89.4	86.9	78.1	96.0	91.8
System-2 Only	5	84.4	81.9	79.5	91.1	87.3
	10	88.1	78.5	79.5	92.0	87.7
	30	86.1	81.3	78.3	98.3	87.9
	50	88.1	80.2	77.1	92.2	88.1
System-1 + System-2	5	88.3	89.1	82.3	95.8	92.2
	10	91.8	90.3	82.3	96.2	93.3
	30	92.9	90.7	79.2	97.1	93.9
	50	92.2	90.3	81.3	96.3	93.4

```
14
15     Examples of INVALID categories:
16     - "Food and Drinks"
17     - "University and Courses"
18
19     Examples of VALID categories:
20     - "Food"
21     - "Drinks"
22     - "University"
23     - "Courses"
24
25 2. Output each category as a dictionary entry where the key is the category name and the
value is a list of node indexes (integers). Only refer to nodes by their indexes. Do not
repeat node names.
26
27     Output format:
28     [
29     {"category": "xx", "indexes": [0, 1, 2, 4]}},
30     {"category": "xxx", "indexes": [2, 3, 4]}
31     ]
32
33     The tag is a list of descriptors (each descriptor maximum 3 words, maximum 5
descriptors) that concisely captures the nature or type of the node.
34
35     Tag example:
36     - Entity name: "Son"
37     - Tag: ["Family member", "Happy kid", "Anime lover"]
38
39 3. A node CAN be assigned to MULTIPLE categories at the same time.
40
41     Key points for multi-category classification:
```

表10: 按问题类型划分的LoCoMo详细性能（LLM作为评判者的分数）

设置	前- <i>k</i>	多跳	时序	开放域	单跳	总体
系统-1 RAG	5	49.6	70.1	65.6	66.8	64.3
	10	64.9	76.6	67.7	76.5	73.8
	30	77.3	82.9	69.8	86.0	82.7
	50	81.6	84.1	71.9	89.1	85.6
系统-1 图	5	79.4	61.4	75.0	86.7	79.4
	10	84.8	62.6	74.0	88.6	81.6
	30	87.6	66.4	77.1	91.6	84.7
	50	90.1	68.5	79.2	92.4	86.2
系统-1 RAG +图	5	81.6	83.2	71.9	91.6	86.8
	10	85.1	84.7	75.0	93.7	89.1
	30	87.9	86.3	76.0	94.9	90.6
	50	89.4	86.9	78.1	96.0	91.8
仅系统-2	5	84.4	81.9	79.5	91.1	87.3
	10	88.1	78.5	79.5	92.0	87.7
	30	86.1	81.3	78.3	98.3	87.9
	50	88.1	80.2	77.1	92.2	88.1
系统-1 + 系统二	5	88.3	89.1	82.3	95.8	92.2
	10	91.8	90.3	82.3	96.2	93.3
	30	92.9	90.7	79.2	97.1	93.9
	50	92.2	90.3	81.3	96.3	93.4

```
14
15     Examples of INVALID categories:
16     - "Food and Drinks"
17     - "University and Courses"
18
19     Examples of VALID categories:
20     - "Food"
21     - "Drinks"
22     - "University"
23     - "Courses"
24
25 2. Output each category as a dictionary entry where the key is the category name and the
value is a list of node indexes (integers). Only refer to nodes by their indexes. Do not
repeat node names.
26
27     Output format:
28     [
29     {"category": "xx", "indexes": [0, 1, 2, 4]}},
30     {"category": "xxx", "indexes": [2, 3, 4]}
31     ]
32
33     The tag is a list of descriptors (each descriptor maximum 3 words, maximum 5
descriptors) that concisely captures the nature or type of the node.
34
35     Tag example:
36     - Entity name: "Son"
37     - Tag: ["Family member", "Happy kid", "Anime lover"]
38
39 3. A node CAN be assigned to MULTIPLE categories at the same time.
40
41     Key points for multi-category classification:
```

```
42 - Each item can be assigned to multiple categories based on shared attributes.
43 - When multiple categories are formed for an item, select the minimal subset of
44   features that are common across the grouped items.
45
46 Examples for different hierarchy levels:
47
48 Layer 1 (specific entities):
49 - "Microsoft Research Asia" and "Microsoft Research Shanghai" share the same parent
50   organization (Microsoft) and a similar research focus (AI). They are grouped under:
51   - "Microsoft Research Labs"
52   - "AI-focused Research Labs"
53
54 - "Microsoft Research Asia" belongs to both "Microsoft Research Labs" and "NLP-focused
55   Labs".
56
57 Layer 2 (category nodes from Layer 1):
58 - "Microsoft Research Labs" belongs to:
59   - "Tech Company Labs"
60   - "AI Research Organizations"
61
62 - "University AI Labs" belongs to:
63   - "Academic Institutions"
64   - "AI Research Organizations"
65
66 Layer 3 (higher-level abstractions):
67 - "Tech Company Labs" belongs to:
68   - "Commercial Organizations"
69   - "Research Institutions"
70
71 - "Academic Institutions" belongs to:
72   - "Educational Organizations"
73   - "Research Institutions"
74
75 Layer 4 (top-level concepts):
76 - "Research Institutions" belongs to:
77   - "Knowledge Organizations"
78
79 - "Commercial Organizations" belongs to:
80   - "Economic Entities"
81
82 4. There must be NO leftover or ungrouped nodes. Single-member categories are allowed if
83   necessary.
84
85 5. The node name "user" and any first-person references ("I", "me") MUST be categorized
86   into one category called "Speaker".
87   """
88
89   guidance = f"""
90   <GUIDANCE ON CATEGORY GRANULARITY>
91   You are performing hierarchical semantic clustering from specific to abstract.
92
93   You are currently at Layer {layer}, where:
94   - Layer 1 contains the most specific, fine-grained categories.
95   - Higher layers should group lower-layer categories into broader, more abstract super-
96     categories.
```

```
42 - Each item can be assigned to multiple categories based on shared attributes.
43
44 Examples for different hierarchy levels:
45
46 Layer 1 (specific entities):
47 - "Microsoft Research Asia" and "Microsoft Research Shanghai" share the same parent
48   organization (Microsoft) and a similar research focus (AI). They are grouped under:
49   - "Microsoft Research Labs"
50   - "AI-focused Research Labs"
51
52 - "Microsoft Research Asia" belongs to both "Microsoft Research Labs" and "NLP-focused
53   Labs".
54
55 Layer 2 (category nodes from Layer 1):
56 - "Microsoft Research Labs" belongs to:
57   - "Tech Company Labs"
58   - "AI Research Organizations"
59
60 - "University AI Labs" belongs to:
61   - "Academic Institutions"
62   - "AI Research Organizations"
63
64 Layer 3 (higher-level abstractions):
65 - "Tech Company Labs" belongs to:
66   - "Commercial Organizations"
67   - "Research Institutions"
68
69 - "Academic Institutions" belongs to:
70   - "Educational Organizations"
71   - "Research Institutions"
72
73 Layer 4 (top-level concepts):
74 - "Research Institutions" belongs to:
75   - "Knowledge Organizations"
76
77 - "Commercial Organizations" belongs to:
78   - "Economic Entities"
79
80 4. There must be NO leftover or ungrouped nodes. Single-member categories are allowed if
81   necessary.
82
83 5. The node name "user" and any first-person references ("I", "me") MUST be categorized
84   into one category called "Speaker".
85   """
86
87   guidance = f"""
88   <GUIDANCE ON CATEGORY GRANULARITY>
89   You are performing hierarchical semantic clustering from specific to abstract.
90
91   You are currently at Layer {layer}, where:
92   - Layer 1 contains the most specific, fine-grained categories.
93   - Higher layers should group lower-layer categories into broader, more abstract super-
94     categories.
```



```
95 - "Golden Retriever", "Poodle", "German Shepherd" -> "Dog breeds"
96 - "Persian Cat", "Siamese Cat" -> "Cat breeds"
97 - "Bengal Tiger", "Siberian Tiger" -> "Tiger subspecies"
98 - "Oak tree", "Pine tree" -> "Tree species"
99
100 Layer 2:
101 - "Dog breeds", "Cat breeds" -> "Pets"
102 - "Dog breeds", "Tiger subspecies" -> "Mammals"
103 - "Tiger subspecies" -> "Wild animals"
104 - "Tree species" -> "Trees"
105
106 Layer 3:
107 - "Pets", "Wild animals" -> "Animals"
108 - "Trees" -> "Plants"
109
110 Layer 4:
111 - "Animals", "Plants" -> "Living organisms"
112
113 Key points:
114 - Categories may belong to multiple parent categories.
115 - Do not merge categories that are too loosely related.
116
117 Your job at Layer {layer}:
118 - Merge semantically similar categories from Layer {layer - 1}.
119 - Each new category should reflect a shared attribute, domain, or higher-level concept.
120 - Multiple category assignments are allowed when justified.
121
122 Previous Layer {layer - 1} categories example:
123 {prev_example}
124 </GUIDANCE ON CATEGORY GRANULARITY>
125 """
126
127     user_prompt = f"""
128 <NODE INDEXED NAMES AND DESCRIPTIONS>
129 {context['content']}
130 </NODE INDEXED NAMES AND DESCRIPTIONS>
131
132 <EXISTING CATEGORIES>
133 These are names and descriptions of categories previously created. Reuse them if
134 applicable.
135 {context['existing_categories']}
136 </EXISTING CATEGORIES>
137
138 {guidance}
139
140 # ATTENTION
141 - The node name "user" and any first-person references ("I", "me") MUST be categorized
142 into one category called "Speaker". If the "Speaker" category does not exist, skip this
143 node.
144 - The category name MUST NOT include the word "and".
145
146 Please follow the INSTRUCTIONS and GUIDANCE carefully to ensure accurate categorization
147 and meaningful hierarchical relationships.
148 DO NOT INCLUDE ANY INVALID CATEGORIES.
149 """
150
151 return [
152     Message(role="system", content=sys_prompt),
153     Message(role="user", content=user_prompt),
```

```
95 - "Golden Retriever", "Poodle", "German Shepherd" -> "Dog breeds"96 - "Persian Cat",
"Siamese Cat" -> "Cat breeds"97 - "Bengal Tiger", "Siberian Tiger" -> "Tiger subspecies"
98 - "Oak tree", "Pine tree" -> "Tree species"99100 Layer 2:101 - "Dog breeds",
"Cat breeds" -> "Pets"102 - "Dog breeds", "Tiger subspecies" -> "Mammals"
103 - "Tiger subspecies" -> "Wild animals"104 - "Tree species" -> "Trees"105106 Layer 3:
107 - "Pets", "Wild animals" -> "Animals"108 - "Trees" -> "Plants"109110 Layer 4:
111 - "Animals", "Plants" -> "Living organisms"112113 Key points:
114 - Categories may belong to multiple parent categories.
115 - Do not merge categories that are too loosely related.116
117 Your job at Layer {layer}:
118 - Merge semantically similar categories from Layer {layer - 1}.
119 - Each new category should reflect a shared attribute, domain, or higher-level concept.
120 - Multiple category assignments are allowed when justified.121
122 Previous Layer {layer - 1} categories example:123 {prev_example}
124 </GUIDANCE ON CATEGORY GRANULARITY>125 """126127     user_prompt = f"""
128 <NODE INDEXED NAMES AND DESCRIPTIONS>129 {context['content']}
130 </NODE INDEXED NAMES AND DESCRIPTIONS>131132 <EXISTING CATEGORIES>
133 These are names and descriptions of categories previously created. Reuse them if
134 applicable.135 {context['existing_categories']}136 </EXISTING CATEGORIES>137 {guidance}
138
139 # ATTENTION
140
141 into one category called "Speaker". If the "Speaker" category does not exist, skip thisnode.
142 - The category name MUST NOT include the word "and".142
143
144 and meaningful hierarchical relationships.144 DO NOT INCLUDE ANY INVALID CATEGORIES.145 """
146147     return [148         Message(role="system", content=sys_prompt),
149         Message(role="user", content=user_prompt),
```

```
150 ]

Below is the instruction to conduct Global Selection.

1  NODE_SELECTION_PROMPT_TEMPLATE = """You are analyzing a hierarchical knowledge graph to
help answer a user query.
2
3  Select all nodes that could help answer the query. A node is helpful if it:
4
5  - Directly relates to the query;
6  - Covers a clearly relevant topic, concept, or category;
7  - Provides useful background or context;
8  - Contains user-specific information (e.g. interests, goals, constraints);
9  - Likely has sub-nodes that may be helpful.
10
11 Do not be overly strict: include nodes that might provide context or personalization, even
if they seem partially redundant.
12
13 For each selected node:
14 - "name" is the node's name.
15 - "uuid" is the node's unique identifier.
16 - "get_all_children" is an boolean value. Set true only if you're confident all its sub-
nodes are helpful.
17 ---
18 User Query:
19 "{query}"
20
21 Available Nodes:
22 {nodes_info}
23 """
```

```
150 ]以下是执行全局选择的指令。
1
help answer a user query.2
3  Select all nodes that could help answer the query. A node is helpful if it:
5  - Directly relates to the query;
7  - Covers a clearly relevant topic, concept, or category;
8  - Provides useful background or context;
9  - Contains user-specific information (e.g. interests, goals, constraints);
11 - Likely has sub-nodes that may be helpful.
12
13 Do not be overly strict: include nodes that might provide context or personalization, even
if they seem partially redundant.
14
15 For each selected node:
16 - "name" is the node's name.
17 - "uuid" is the node's unique identifier.
18 - "get_all_children" is an boolean value. Set true only if you're confident all its sub-
nodes are helpful.
19 ---
20 User Query:
21 "{query}"
22
23 Available Nodes:
24 {nodes_info}
```